

Plan de Gestión del Proyecto de Software (SPMP)

SecureTicket

Plataforma Web2.5 de boletería con NFTs en Stellar/Soroban

Versión 1.5 — 21 de mayo de 2026



Pontificia Universidad Javeriana Bogotá

Planeación Proyecto Final

Director: Rafael Vicente Páez Méndez, PhD

Autores (Grupo 12):

Juan Diego Arias Durán

Santiago Andrés Mesa Niño

Andrés Felipe Manosalva Garzón

Juan Camilo Carvajal Rodríguez

Semestre 2026-1

Historial de cambios

Versión	Fecha	Descripción	Autor(es)
1.0	28 de enero de 2026	Primera emisión del SPMP con secciones 1 a 6: propósito, alcance preliminar, objetivos, supuestos, restricciones, entregables y borrador de calendarización y presupuesto.	Grupo 12
1.1	18 de febrero de 2026	Refinamiento del documento tras la fase de análisis: ajuste de propósito y alcance del producto, ampliación del glosario con términos clave (PM, WIP, DoD, Kanban) y refinamiento de la estructura de las tablas de presupuesto.	Grupo 12
1.2	12 de marzo de 2026	Alineación con el SRS y el SDD preliminar (vistas C4, secuencias y modelo de datos on-chain/off-chain): se incorporan los tres contratos Soroban como entregables (<code>event_contract</code> , <code>factory_contract</code> , <code>ticket_nft_contract</code>) y se actualizan los objetivos.	Grupo 12
1.3	9 de abril de 2026	Actualización con el avance del backend (Express + Prisma + Indexer) y el frontend (React/Vite + Freightier): se ajustan responsables y horas-persona por integrante; se documenta el patrón XDR proxy <i>stateless</i> ; se reorganizan los entregables en software y documentación.	Grupo 12
1.4	30 de abril de 2026	Incorporación de la Phase 4 (panel de administración y rol STAFF/verificadores on-chain) en el alcance; actualización de la carta Gantt y de los hitos cumplidos hasta semana 13; incorporación de glosario Web2.5 (Freightier, indexer, XDR proxy, SEP-41, burn/remint, STAFF).	Grupo 12

Versión	Fecha	Descripción	Autor(es)
1.5	21 de mayo de 2026	Cierre retrospectivo del SPMP: alineación final con el sistema entregado (12/12 eventos desplegados en <i>testnet</i>), consolidación de la lista de entregables académicos finales (Memoria de TG, Plan de Proyecto, SRS, SDD, Propuesta de TG y Formato de Biblioteca), ajuste de horas-persona ejecutadas y presupuesto operativo final (servicios públicos del equipo).	Grupo 12

Prefacio

Este **Plan de Gestión del Proyecto de Software (SPMP)** documenta, de forma retrospectiva, cómo se dirigió y controló la construcción de **SecureTicket** (Plataforma Web2.5 de boletería con NFTs en Stellar/Soroban). El documento cubre el ciclo de vida adoptado, la organización del equipo, la calendarización ejecutada, el presupuesto, los riesgos materializados, las prácticas de calidad, la gestión de configuración, la aceptación y la transición del producto, con el fin de dejar un registro trazable y medible del trabajo realizado durante el semestre académico 2026-1.

Audiencia. Equipo del proyecto, director de trabajo de grado, docentes evaluadores y revisores académicos de la Pontificia Universidad Javeriana.

Uso del documento. La **Sección 0.5** ofrece la vista general del producto entregado (visión, propósito, alcance, supuestos, restricciones, entregables y resumen de calendario y presupuesto). Las secciones 8 a 12 detallan ciclo de vida y herramientas, administración del proyecto (estimación, WBS, Gantt, recursos), monitoreo y control (métricas y KPIs), entrega del producto y procesos de soporte (riesgos, configuración, calidad). El enfoque de trabajo fue **Ágil con tablero Kanban** (flujo continuo, WIP limitado, ceremonias ligeras y revisiones semanales con el director).

Relación con artefactos. Este SPMP se coordina con los demás documentos del trabajo de grado: Memoria de TG, Propuesta de TG, Especificación de Requisitos de Software (SRS), Especificación del Diseño (SDD), Formato de Biblioteca y el Demo final.

Convenciones. Terminología consistente: plataforma de boletería, evento, ticket/boleto, NFT, contrato inteligente, Stellar, Soroban, Freightier Wallet, indexer, reventa atómica (burn/remint), verificación on-chain, testnet, KPI, incidente, cambio de alcance.

Mantenimiento. El equipo mantiene el documento y su historial de cambios bajo control de versiones (Git). Las actualizaciones relevantes (alcance, hitos, presupuesto, riesgos críticos) se registraron en la siguiente revisión semanal del equipo con el director.

Índice general

0.1. Introducción	1
0.1.1. Propósito del documento	1
0.1.2. Audiencia y lectores	1
0.1.3. Alcance del documento	1
0.1.4. Relación con otros artefactos	2
0.1.5. Cómo usar este documento	2
0.2. Definiciones, acrónimos y abreviaturas	2
0.3. Descripción general del plan	5
0.4. Políticas y convenciones del documento	5
0.5. Vista general del proyecto	6
0.5.1. Visión del producto	6
0.5.2. Propósito, alcance y objetivos	7
0.5.3. Supuestos y restricciones	8
0.5.4. Entregables	9
0.5.5. Resumen de calendarización y presupuesto	10
0.5.5.1. Calendarización de fases e hitos (semestre 2026-1)	10
0.5.5.2. Presupuesto consolidado	12
0.5.6. Evolución del plan	14
0.6. Contexto del proyecto	15
0.6.1. Modelo de ciclo de vida	15
0.6.1.1. Fases del ciclo de vida ejecutadas	15
0.6.1.2. Prácticas Kanban aplicadas	16
0.6.2. Métodos, lenguajes y herramientas	16
0.6.2.1. Lenguajes de programación	16
0.6.2.2. Herramientas técnicas	16
0.6.2.3. Métodos y técnicas	18
0.6.3. Plan de aceptación del producto	18
0.6.3.1. Niveles de validación	18
0.6.3.2. Criterios de aceptación globales	18
0.6.3.3. Procedimiento de aceptación	19
0.6.4. Estructura organizacional	19

0.6.4.1.	Interfaces externas	19
0.6.4.2.	Estructura interna del equipo	20
0.6.4.3.	Matriz de responsabilidades (RACI resumida)	20
0.6.4.4.	Canales de comunicación	21
0.7.	Administración del proyecto	22
0.7.1.	Métodos y herramientas de estimación	22
0.7.1.1.	Enfoque general	22
0.7.1.2.	Herramientas de estimación y seguimiento	23
0.7.1.3.	Calibración y aprendizajes	23
0.7.2.	Plan de inicio del proyecto	23
0.7.2.1.	Entrenamiento y nivelación	24
0.7.2.2.	Infraestructura inicial	24
0.7.3.	Planes de trabajo	25
0.7.3.1.	Estructura desglosada del trabajo (WBS)	25
0.7.3.2.	Carta Gantt ejecutada	27
0.7.3.3.	Asignación de recursos	27
0.7.3.4.	Presupuesto detallado	28
0.8.	Monitoreo y control del proyecto	29
0.8.1.	Administración de requerimientos	29
0.8.1.1.	Origen y trazabilidad de los requerimientos	29
0.8.1.2.	Proceso de cambio de requerimientos	30
0.8.1.3.	Cambios de alcance materializados	30
0.8.2.	Monitoreo y control de progreso	31
0.8.2.1.	Ciclo de monitoreo	31
0.8.2.2.	KPIs definidos y resultados	32
0.8.2.3.	Métricas auxiliares de seguimiento	33
0.8.2.4.	Acciones correctivas aplicadas	33
0.8.3.	Cierre de fases e iteraciones	34
0.8.3.1.	Criterios de cierre	34
0.8.3.2.	Ritual de cierre de fase	34
0.8.3.3.	Lecciones aprendidas consolidadas	34
0.8.3.4.	Cierre formal del proyecto	35
0.9.	Entrega del producto	35
0.9.1.	Estrategia de entrega	35
0.9.2.	Artefactos a entregar	36
0.9.3.	Cronograma de entrega	37
0.9.4.	Capacitación y soporte al receptor	37
0.9.4.1.	Capacitación durante la sustentación	38
0.9.4.2.	Material de apoyo	38

0.9.4.3. Soporte post-entrega	38
0.9.5. Criterios de aceptación de la entrega	38
0.10. Procesos de soporte	39
0.10.1. Ambiente de trabajo y reglas del equipo	39
0.10.1.1. Ambiente de trabajo	39
0.10.1.2. Reglas del equipo	39
0.10.2. Análisis y administración de riesgos	40
0.10.2.1. Proceso de gestión de riesgos	40
0.10.2.2. Registro de riesgos del proyecto	40
0.10.2.3. Aprendizajes sobre gestión de riesgos	42
0.10.3. Administración de configuración y documentación	42
0.10.3.1. Identificación de elementos de configuración	42
0.10.3.2. Control de versiones	43
0.10.3.3. Control de cambios	43
0.10.3.4. Gestión de versiones de los documentos	43
0.10.3.5. Respaldos	44
0.10.4. Métricas y proceso de medición	44
0.10.4.1. Proceso de medición	44
0.10.4.2. Categorías de métricas	44
0.10.4.3. Fuentes de datos	45
0.10.5. Control de calidad	45
0.10.5.1. Verificación	45
0.10.5.2. Validación	45
0.10.5.3. Revisiones e inspecciones	46
0.10.5.4. Definition of Done (DoD)	46
0.10.5.5. Resumen de resultados de calidad	46
0.11. Cierre del proyecto	46
0.11.1. Balance final del proyecto	47
0.11.2. Cumplimiento de objetivos	47
0.11.3. Lecciones aprendidas globales	48
0.11.4. Riesgos residuales y trabajo futuro	49
0.11.5. Cierre formal del SPMP	49
0.12. Referencias	50

Índice de cuadros

3.	Resumen de fases e hitos ejecutados durante el semestre 2026-1	11
4.	Presupuesto en horas-persona por integrante (semestre 2026-1)	13
5.	Presupuesto monetario consolidado (costos operativos)	13
6.	Herramientas técnicas del proyecto	17
7.	Roles y responsabilidades del equipo	20
8.	Matriz RACI por entregable principal	21
9.	Correspondencia T-shirt sizing $\leftrightarrow$ horas-persona	23
10.	Carta Gantt ejecutada (semanas 1–16 del semestre 2026-1)	27
11.	Asignación de recursos por paquete WBS (% del esfuerzo del integrante) .	27
12.	Horas-persona ejecutadas por paquete WBS	28
13.	Presupuesto monetario desagregado por fase y rubro	29
14.	Cambios de alcance aprobados durante el semestre	31
15.	KPIs del proyecto: definición, umbral y resultado final	32
16.	Artefactos del paquete de entrega final	36
17.	Cronograma de entrega final	37
18.	Matriz cualitativa de exposición al riesgo	40
19.	Registro de riesgos del proyecto	41
20.	Cumplimiento de objetivos del proyecto	47

0.1. Introducción

0.1.1. Propósito del documento

Documentar el *plan de gestión* ejecutado para construir **SecureTicket**, una **plataforma Web2.5 de boletería** que combina la experiencia de usuario tradicional (catálogo de eventos, carrito, *checkout* con pago fiat simulado y cuenta de usuario) con una capa **opcional** de propiedad Web3 implementada como **NFTs SEP-41** sobre la red **Stellar** mediante contratos inteligentes **Soroban**. El plan describe los procesos, roles, artefactos y mecanismos de control que se aplicaron durante el semestre académico **2026-1** (enero – 24 de mayo de 2026) para entregar un prototipo funcional desplegado en *testnet*, junto con la documentación académica requerida.

0.1.2. Audiencia y lectores

- Miembros del equipo de proyecto (Grupo 12).
- Director de trabajo de grado: **Rafael Vicente Páez Méndez, PhD**.
- Docentes evaluadores y jurados del curso *Planeación Proyecto Final* y del programa de Ingeniería de Sistemas.
- Revisores académicos de la Pontificia Universidad Javeriana.

0.1.3. Alcance del documento

Este SPMP abarca:

- **Vista general** (Sección 0.5): visión del producto, alcance entregado, objetivos cumplidos, supuestos, restricciones, entregables y resumen de calendarización y presupuesto.
- **Contexto del proyecto** (Sección 8): modelo de ciclo de vida ágil/Kanban, lenguajes y herramientas (Rust/Soroban, Node/Express/Prisma, React/Vite, Supabase, Vercel, Railway, Freightier Wallet), plan de aceptación del producto y organización del equipo.
- **Administración del proyecto** (Sección 9): métodos de estimación, plan de inicio, WBS, carta Gantt, asignación de recursos y presupuesto detallado.
- **Monitoreo y control** (Sección 10): administración de requerimientos, métricas de progreso, KPIs (cobertura de pruebas, eventos desplegados en *testnet*, defectos abiertos) y cierre de fases.
- **Entrega del producto** (Sección 11): plan de transición y entrega al director y jurados.
- **Procesos de soporte** (Sección 12): análisis y administración de riesgos, gestión de configuración, métricas y control de calidad.

0.1.4. Relación con otros artefactos

Este plan se integra con los siguientes documentos del trabajo de grado:

- **Memoria de TG:** documento principal del trabajo de grado que sintetiza problemática, marco teórico, metodología, resultados y conclusiones.
- **Propuesta de TG:** anteproyecto aprobado que define el alcance académico del trabajo.
- **SRS (Especificación de Requisitos de Software):** requisitos funcionales y no funcionales, historias de usuario y criterios de aceptación.
- **SDD (Especificación del Diseño):** vistas arquitectónicas C4, modelo de datos on-chain/off-chain, diagramas de secuencia y atributos de calidad.
- **Formato de Biblioteca:** formato institucional de entrega del trabajo de grado a la biblioteca de la Universidad.
- **Demo final:** producto desplegado en *testnet* presentado en vivo ante el director y los jurados, o bien como **video** grabado del recorrido completo de la plataforma, junto con los enlaces públicos al repositorio del código fuente.

Las decisiones de gestión descritas en este SPMP gobiernan la creación, revisión y mantenimiento de todos los artefactos listados.

0.1.5. Cómo usar este documento

- Para una **visión rápida del producto entregado:** leer Sección [0.5](#).
- Para consultar **procesos, prácticas, herramientas y métricas:** revisar Secciones 8 a 12.
- Utilizar la **Tabla de Contenidos**, **Lista de Figuras** y **Lista de Tablas** para navegación directa.
- Los **hipervínculos internos** (azul) facilitan referencias cruzadas entre secciones, tablas y figuras.

0.2. Definiciones, acrónimos y abreviaturas

Término	Definición
Web2.5	Modelo híbrido que combina la experiencia tradicional Web2 (catálogo, carrito, checkout, cuenta de usuario) con una capa Web3 opcional (NFT, billetera, firma criptográfica). El usuario puede operar sin billetera; la blockchain es una capa de aseguramiento adicional.
NFT	<i>Non-Fungible Token</i> : activo digital único y trazable que representa un boleto en blockchain. Cada NFT tiene un identificador único que impide duplicación o falsificación.
SEP-41	<i>Stellar Ecosystem Proposal 41</i> : estándar de <i>token</i> en Soroban utilizado por el contrato <code>ticket_nft_contract</code> para que Freighters clasifique los boletos como <i>Collectibles</i> .
Contrato inteligente	Programa autoejecutable desplegado en Soroban (Stellar) que gobierna la emisión, transferencia, comisiones, reventa y verificación de los boletos tokenizados.
Stellar	Red blockchain descentralizada diseñada para pagos rápidos y emisión de activos digitales. Utiliza el Protocolo de Consenso Stellar (SCP).
Soroban	Plataforma de <i>smart contracts</i> sobre Stellar basada en WebAssembly (Wasm), que permite programar lógica de negocio en Rust.
Freighters Wallet	Extensión de navegador (billetera no-custodial) para Stellar. Los usuarios firman transacciones localmente; el backend nunca custodia llaves privadas.
XDR proxy	Patrón aplicado en el backend: construye <i>transaction envelopes</i> (XDR) y los devuelve al frontend para que el usuario firme con su billetera, sin que el servidor tenga acceso a la llave privada.
Indexer	Proceso en segundo plano (cada 5 s) que consulta el RPC de Soroban, traduce eventos on-chain a actualizaciones en PostgreSQL y mantiene sincronizadas las vistas off-chain.
Boleto/Ticket	Entrada a un evento. Existe siempre como registro off-chain (PostgreSQL) y opcionalmente como NFT on-chain cuando el usuario decide “asegarlo” en blockchain.
Burn/Remint	Mecanismo de reventa atómica: el NFT del vendedor se quema (<i>burn</i>) y se miente (<i>remint</i>) una nueva versión a nombre del comprador en la misma transacción, evitando estados inconsistentes.

Término		Definición
Reventa P2P	atómica	Operación de mercado secundario <i>peer-to-peer</i> ejecutada en un solo contrato Soroban: cobra comisiones (organizador + plataforma), paga al vendedor y reasigna el NFT al comprador.
Verificador / STAFF		Personal autorizado por el organizador para validar boletos en la puerta del evento. Registrado on-chain (<code>Verificador(Address)</code>) y off-chain (rol STAFF en PostgreSQL).
Redención on-chain		Marcado irreversible del boleto como usado (<code>usado=true</code>) mediante <code>redimir_boleto</code> ; solo verificadores autorizados pueden invocarla.
Phase 4		Fase del proyecto que incorporó el panel de administración, gestión de eventos y roles STAFF, integrando catálogo Web2 y operación on-chain.
Testnet		Red de pruebas de Stellar sin valor monetario real, utilizada para validar contratos, transacciones y flujos <i>end-to-end</i> antes de producción.
KPI		<i>Key Performance Indicator</i> : indicador clave de desempeño (ej: % de pruebas aprobadas, defectos abiertos, cumplimiento de hitos).
BPMN		<i>Business Process Model and Notation</i> : notación estándar para modelar procesos de negocio.
Carta Gantt		Diagrama de barras que visualiza actividades del proyecto, duraciones, dependencias y línea temporal.
Kanban		Método ágil de flujo continuo con tablero visual, WIP limitado y revisiones periódicas; aplicado como marco de trabajo del equipo.
WIP		<i>Work In Progress</i> : límite de trabajo en curso para evitar sobrecarga y mejorar el flujo.
DoD		<i>Definition of Done</i> : criterios que una tarea o historia debe cumplir para considerarse completada.
E2E		<i>End-to-End</i> : flujo completo desde la creación del evento hasta la verificación del boleto en puerta.
PM		<i>Project Manager</i> : rol responsable de coordinación, seguimiento y comunicación con el director.

0.3. Descripción general del plan

Este plan se estructura en las siguientes secciones principales:

- **Sección 0.5 (Vista general del proyecto):** visión del producto entregado, propósito, alcance, objetivos, supuestos y restricciones, entregables y resumen de calendarización y presupuesto.
- **Sección 8 (Contexto del proyecto):** modelo de ciclo de vida ágil con tablero Kanban, lenguajes y herramientas (Rust/Soroban, Node/Express/Prisma, React/Vite, Supabase, Vercel, Railway, Freightier Wallet), plan de aceptación y estructura organizacional del equipo.
- **Sección 9 (Administración del proyecto):** métodos y herramientas de estimación; plan de inicio (entrenamiento en Soroban y configuración de infraestructura); planes de trabajo detallados (WBS, carta Gantt ejecutada, asignación de recursos y presupuesto).
- **Sección 10 (Monitoreo y control):** administración de requerimientos; seguimiento de progreso con métricas y KPIs (cobertura de pruebas, eventos desplegados, defectos abiertos); cierre de fases e iteraciones.
- **Sección 11 (Entrega del producto):** plan de transición y entrega al director y jurados; responsables y cronograma de artefactos finales.
- **Sección 12 (Procesos de soporte):** ambiente de trabajo y reglas de equipo; análisis y administración de riesgos materializados; gestión de configuración (Git, migraciones Prisma, despliegues); métricas y proceso de medición; control de calidad (verificación, validación, revisiones).

0.4. Políticas y convenciones del documento

- **Control de cambios y versión:** toda actualización relevante (cambios de alcance, hitos, presupuesto, riesgos críticos, decisiones arquitectónicas) se registra en el **Historial de Cambios** con: número de versión, fecha, descripción y autor(es) responsable(s).
- **Estructura y estilos:** el documento utiliza jerarquía de títulos (`\section`, `\subsection`, `\subsubsection`) con numeración automática de L^AT_EX. Figuras y tablas incluyen título descriptivo y fuente cuando aplique. Tamaño de letra en figuras/tablas: mínimo 10pt.
- **Lenguaje y enlaces:** redacción en español con terminología técnica consistente (ver Sección 0.2). Hipervínculos internos (azul) para referencias cruzadas entre secciones, tablas y figuras. URLs externas en color azul.

- **Trazabilidad:** cada plan, métrica o decisión referencia los artefactos relacionados (SRS, SDD, Memoria de TG) y las secciones del SPMP donde se detallan.
- **Revisiones periódicas:** reunión semanal de seguimiento (ver Sección 0.5.6). El equipo sincroniza cambios de alcance, actualizaciones de riesgos y ajustes al plan.
- **Formato no alterable:** mantener tamaños de letra, márgenes, espaciado y estilos definidos. Permitido: cambio de colores (sin dificultar legibilidad) y marcas de agua institucionales.

0.5. Vista general del proyecto

0.5.1. Visión del producto

SecureTicket es una **plataforma Web2.5 de boletería** para eventos en Colombia, construida durante el semestre 2026-1 como prototipo funcional desplegado en *testnet*. Combina la experiencia tradicional Web2 (catálogo de eventos, carrito, *checkout* con pago fiat simulado y cuenta de usuario) con una capa Web3 **opcional** de propiedad on-chain: el boleto se representa como NFT SEP-41 en Soroban/Stellar, soporta reventa atómica *peer-to-peer* mediante *burn/remint* y se verifica en puerta por verificadores autorizados on-chain.

La tesis subyacente es que blockchain puede **mitigar problemas estructurales del mercado tradicional de boletería** (reventa abusiva, falsificación, opacidad de comisiones) **sin sacrificar UX masiva**: el flujo de compra Web2 (carrito → checkout → orden → boleto) funciona sin billetera; “asegurar en blockchain” es una acción adicional opt-in.

Beneficios entregados:

- **Trazabilidad opt-in:** historial inmutable de propiedad para boletos asegurados on-chain.
- **Autenticidad criptográfica:** imposibilidad de duplicación o falsificación de boletos NFT.
- **Comisiones embebidas:** reglas de regalías programadas en el contrato (5 % organizador, 3 % plataforma en cada reventa).
- **Reventa atómica:** *burn/remint* en una sola transacción que evita estados inconsistentes.
- **No-custodial:** el backend nunca custodia llaves privadas; el usuario firma con Freighter Wallet.
- **Verificación en puerta:** sólo verificadores autorizados (registrados on-chain) pueden redimir boletos.

0.5.2. Propósito, alcance y objetivos

Propósito

Construir un **prototipo académico funcional** que evidencie la viabilidad técnica de una plataforma de boletería Web2.5, integrando catálogo y carrito tradicionales con contratos inteligentes Soroban para la emisión de NFTs SEP-41, reventa atómica con comisiones programables y verificación on-chain en puerta. El prototipo se desplegó en la *testnet* de Stellar y constituye la demostración del modelo ante el director y los jurados del trabajo de grado.

Alcance (qué incluye el proyecto entregado)

- **Capa Web2 (frontend + backend):** catálogo público de eventos con búsqueda y filtros (categoría, ciudad), detalle de evento, carrito de compras, *checkout* con pago fiat simulado, cuenta de usuario con autenticación (*bcrypt* + JWT), órdenes, boletos y vinculación opcional de billetera Freightier. Incluye panel de administración (Phase 4) para organizadores y administradores, y rol STAFF para verificadores de puerta.
- **Contratos inteligentes Soroban (Rust, soroban-sdk 23):**
 - **event_contract** (~1142 LoC, 35+ tests): ciclo de vida del boleto, listado/cancelación, compra primaria (100 % al organizador), reventa con *burn/remint* y comisiones, gestión de verificadores, redención e invalidación.
 - **factory_contract** (~431 LoC, 13 tests): patrón *Factory* que despliega un **event_contract** independiente por evento.
 - **ticket_nft_contract** (SEP-41, 10 tests, 12 KB WASM): NFT que clasifica el boleto como *Collectible* en Freightier.
- **Backend (Node + Express + Prisma + Indexer):** API REST completa (catálogo, auth, carrito, *checkout*, órdenes, boletos, operaciones on-chain); constructor de XDR *stateless* (XDR proxy) que nunca custodia llaves; indexer que sincroniza PostgreSQL con eventos on-chain cada 5 s.
- **Frontend (React + Vite):** SPA responsive con flujo E2E (navegación, compra, asegurar en blockchain, reventa P2P, escáner QR de verificación); integración con Freightier Wallet para firma de transacciones.
- **Despliegue en testnet:** 12/12 eventos con **event_contract** + **ticket_nft_contract** desplegados; wallets ADMIN, ORGANIZER, PLATFORM, BUYER1/2 y VERIFIER configuradas.
- **Documentación académica:** Memoria de TG, este SPMP (Plan de Proyecto), SRS, SDD, Propuesta de TG, Formato de Biblioteca y Demo final.

Alcance (qué NO incluye el proyecto)

- Despliegue en **mainnet** de Stellar con fondos reales.
- Pasarelas de pago fiat reales (Stripe, PayU, etc.); el *checkout* es simulado.
- Aplicaciones móviles nativas iOS/Android (sólo web responsive).
- Integración con sistemas de boletería en producción.
- Modo de verificación *offline* en puerta (decisión explícita de no implementar).
- KYC/AML, antifraude avanzado o cumplimiento regulatorio colombiano.

Objetivos cumplidos del proyecto

1. **Modelar** el dominio de boletería Web2.5 y la arquitectura del sistema (diagramas C4 de contexto, contenedores y despliegue; secuencias de reventa atómica y verificación; modelo de datos on-chain/off-chain).
2. **Implementar** la suite de contratos Soroban (`event_contract`, `factory_contract`, `ticket_nft_contract`) con cobertura de pruebas unitarias e integración.
3. **Construir** el backend Express + Prisma con XDR proxy *stateless*, indexer e integración a Soroban RPC y Horizon.
4. **Construir** el frontend React + Vite con flujo E2E completo, integración con Freightier y escáner QR.
5. **Desplegar** 12 eventos demostrativos en *testnet* con sus respectivos contratos y NFTs.
6. **Validar** el sistema contra los criterios de aceptación del SRS mediante pruebas unitarias en Rust, de integración en Node y E2E desde el frontend.
7. **Producir** la documentación académica del trabajo de grado: Memoria de TG, Plan de Proyecto (este SPMP), SRS, SDD, Propuesta de TG y Formato de Biblioteca.

0.5.3. Supuestos y restricciones

Supuestos

- Acceso continuo y estable a la *testnet* de Stellar y al RPC público de Soroban durante todo el semestre.
- Disponibilidad de herramientas gratuitas o con plan académico: GitHub, Vercel (frontend), Railway/Render (backend), Supabase (PostgreSQL), Freightier Wallet.
- Equipo de 4 estudiantes disponible durante el semestre 2026-1 con dedicación parcial compatible con la carga académica.

- Calendario académico alineado: cierre del semestre y entrega del paquete documental el **24 de mayo de 2026**; sustentación ante jurados en la ventana **25 de mayo – 5 de junio de 2026** (fecha exacta por confirmar por la coordinación).
- Acceso a documentación técnica actualizada de Stellar, Soroban SDK 23, Prisma y demás dependencias.
- Disponibilidad del director (Rafael V. Páez) para revisiones semanales.

Restricciones

- **Tiempo acotado:** semestre académico 2026-1 (aprox. 16 semanas; cierre y entrega documental el 24 de mayo de 2026; sustentación entre el 25 de mayo y el 5 de junio de 2026 según asignación de la coordinación).
- **Cumplimiento institucional:** ética académica, buenas prácticas de desarrollo y estándares de documentación de la Pontificia Universidad Javeriana.
- **Naturaleza académica:** prototipo demostrativo, no apto para uso comercial en main-net.
- **Recursos humanos:** 4 integrantes con dedicación parcial.
- **Recursos financieros:** presupuesto operativo mínimo (hosting gratuito, impresiones, transporte); sin inversión en infraestructura productiva.
- **Tecnologías obligadas:** Stellar/Soroban como plataforma blockchain (alcance temático del trabajo de grado).

0.5.4. Entregables

Los entregables del proyecto se clasifican en artefactos de software y artefactos de documentación.

Artefactos de software (código fuente y despliegue):

1. **Suite de contratos inteligentes Soroban (Rust):**
 - `event_contract` desplegado en *testnet* (WASM hash 32fbb9bf...b0ef).
 - `factory_contract` para despliegue por evento.
 - `ticket_nft_contract` SEP-41 (WASM hash 04bcf7f1...8a70).
 - Pruebas unitarias e integración (35+, 13 y 10 respectivamente).
2. **Backend (Express + Prisma + Indexer):** servicios REST para catálogo, autenticación, carrito, *checkout*, órdenes, boletos y operaciones on-chain; XDR proxy *stateless*; indexer Soroban; esquema PostgreSQL versionado con Prisma.

3. **Frontend (React + Vite):** SPA responsive con flujo E2E, integración Freightier y escáner QR.
4. **Despliegue demostrativo:** 12 eventos en *testnet* de Stellar con sus contratos asociados; frontend en Vercel; backend en Railway; base de datos en Supabase.

Artefactos de documentación académica (entrega final): Al cierre del trabajo de grado se entregan formalmente los siguientes **seis documentos** a la Universidad:

1. **Memoria de TG:** documento principal del trabajo de grado.
2. **Plan de Proyecto (este SPMP):** Plan de Gestión del Proyecto de Software, con versionado v1.0 a v1.5.
3. **SRS:** Especificación de Requisitos de Software.
4. **SDD:** Especificación del Diseño (vistas C4, secuencias, modelo de datos on-chain/off-chain, atributos de calidad).
5. **Propuesta de TG:** anteproyecto del trabajo de grado.
6. **Formato de Biblioteca:** formato institucional para la entrega del TG a la biblioteca de la Universidad.

Demo y código fuente (no se entregan como documento):

- El **Demo final** (sistema desplegado en *testnet*) se presenta en vivo ante el director y los jurados, o bien como un **video** grabado del recorrido completo de la plataforma; no se anexa como artefacto descargable al paquete documental.
- Del **código fuente** sólo se entregan los **enlaces públicos al repositorio** que lo aloja; no se anexa el código al paquete documental.

0.5.5. Resumen de calendarización y presupuesto

0.5.5.1. Calendarización de fases e hitos (semestre 2026-1)

La Tabla 3 resume las fases ejecutadas, sus hitos/entregables clave, fechas reales y responsables principales. El semestre se desarrolló entre la última semana de enero de 2026 y el cierre y entrega documental el **24 de mayo de 2026**. La sustentación ante jurados se realiza en una fecha asignada por la coordinación dentro de la ventana **25 de mayo – 5 de junio de 2026**.

Cuadro 3: Resumen de fases e hitos ejecutados durante el semestre 2026-1

Fase	Hito / Entregable clave	Fechas (2026)	Responsable principal
Inicio y Alcance	SPMP v1.0 (propósito, alcance, objetivos, riesgos iniciales, WBS preliminar); configuración del repositorio y tablero Kanban	Sem. 1–2 (ene 26 – feb 8)	PM (Andrés F. Manosalva)
Análisis y Diseño	SRS preliminar; SDD v0.1–0.3 (vistas C4, secuencias, modelo de datos); definición de criterios de aceptación	Sem. 3–5 (feb 9 – mar 1)	PM + Blockchain + Backend
Implementación de contratos	<code>event_contract</code> , <code>factory_contract</code> y <code>ticket_nft_contract</code> con pruebas unitarias; despliegue inicial en <i>testnet</i>	Sem. 6–9 (mar 2 – mar 29)	Blockchain (Santiago Mesa)
Backend + Indexer	API REST (catálogo, auth, carrito, checkout, órdenes, boletos); XDR proxy; indexer Soroban (5 s); esquema Prisma	Sem. 7–11 (mar 9 – abr 12)	Backend (Juan Diego Arias)
Frontend + Freighter	SPA React/Vite, integración Freighter, escáner QR, flujo E2E (compra, asegurar, venta, verificación)	Sem. 9–12 (mar 23 – abr 19)	Frontend (Juan Camilo Carvajal)
Phase 4 (Panel admin + STAFF)	Panel de administración, gestión de eventos, rol STAFF/verificadores on-chain	Sem. 12–13 (abr 13 – abr 26)	Backend + Frontend

Continúa en la siguiente página

(Tabla 3 – continuación)

Fase	Hito / Entregable clave	Fechas (2026)	Responsable principal
Pruebas y Validación	Ejecución de casos de prueba unitarios, integración y E2E; evidencias documentadas; despliegue final de 12/12 eventos en <i>testnet</i>	Sem. 14–15 (abr 27 – may 10)	QA/DevOps (Juan Camilo) + todos
Documentación final	Cierre de Memoria de TG, SDD, SRS, SPMP v1.5, Propuesta de TG y Formato de Biblioteca; preparación del Demo final	Sem. 15–16 (may 11 – may 23)	Todos (PM coordina)
Cierre y entrega	Entrega final del paquete documental a la coordinación; <i>post-mortem</i> interno del equipo	Sem. 16 (24 may)	PM
Sustentación	Presentación ante director y jurados; atención a observaciones	25 may – 5 jun (por confirmar)	Todos

0.5.5.2. Presupuesto consolidado

Horas-persona invertidas (estimación retrospectiva) La Tabla 4 detalla la distribución de esfuerzo del equipo durante el semestre, asignada por integrante según su rol predominante. El total de **~255 horas-persona** corresponde a una dedicación aproximada de 16 h/semana por integrante durante 16 semanas, descontando reuniones y semanas con carga académica reducida.

Integrante	Rol predominante	Horas
Andrés Felipe Manosalva Garzón	PM / Coordinación y apoyo transversal	60
Santiago Andrés Mesa Niño	Blockchain (Soroban, Rust)	65
Juan Diego Arias Durán	Backend (Express, Prisma, Indexer) + arreglos Frontend	65
Juan Camilo Carvajal Rodríguez	Frontend (React/Vite) + QA/DevOps	65
Total		255

Cuadro 4: Presupuesto en horas-persona por integrante (semestre 2026-1)

Costos operativos (COP) La Tabla 5 consolida los gastos operativos del proyecto. Se incluye una contingencia del 10 % para imprevistos.

Ítem	Monto (COP)	Comentario
Hosting demo	0	Vercel, Railway y Supabase en planes <i>free tier</i> académicos
Impresiones (anexos)	120,000	Material impreso para la sustentación final
Transporte / viáticos	80,000	Reuniones presenciales con el director y sustentación final
Servicios de luz (4 hogares)	480,000	Consumo eléctrico asociado al trabajo en casa de los 4 integrantes durante ~4 meses (~30.000 COP/mes/integrante)
Subtotal	680,000	
Contingencia 10 %	68,000	Reserva para imprevistos
Total	748,000	

Cuadro 5: Presupuesto monetario consolidado (costos operativos)

Notas sobre el presupuesto:

- Las licencias de software son ~0 COP (herramientas *open-source* y planes gratuitos: GitHub, Vercel, Railway, Supabase, Freightier).
- La *testnet* de Stellar no genera costos de transacción reales.

- No se incluyen costos de hardware (equipos personales del equipo).
- Los servicios de luz se estiman como costo prorrateado del consumo eléctrico de los equipos de cada integrante en sus respectivas residencias durante el semestre.
- El presupuesto total de **748.000 COP** es indicativo y se asume cubierto por aportes del propio equipo.

0.5.6. Evolución del plan

El SPMP es un **documento vivo** que se revisa y actualiza periódicamente. El proceso de evolución se gestiona así:

Frecuencia de revisión

- **Revisión semanal:** reunión de seguimiento del equipo (duración: 30–60 min) donde se valida:
 - Avance vs. plan (comparación con carta Gantt).
 - Estado de riesgos (identificación de nuevos riesgos o cambios en prioridad).
 - Necesidad de ajustes al alcance, calendario o presupuesto.
- **Revisión al final de cada fase:** el equipo consolida lecciones aprendidas y actualiza secciones relevantes del SPMP (Sección 0.5.5, Sección 12.2 Riesgos, etc.).

Triggers para actualización del SPMP

El documento se actualiza formalmente (nueva versión en historial de cambios) cuando ocurra alguno de los siguientes eventos:

- **Cambios de alcance:** adición/eliminación de funcionalidades o entregables aprobados por el equipo y el director.
- **Ajustes a hitos o fechas críticas:** reprogramación de entregas debido a retrasos, cambios en calendario académico o decisiones estratégicas.
- **Modificaciones al presupuesto:** incremento/reducción de costos operativos o reasignación de horas-persona entre roles.
- **Riesgos críticos materializados:** cuando un riesgo de alta prioridad se convierte en incidente y requiere plan de mitigación documentado.
- **Decisiones arquitectónicas mayores:** cambios en la pila tecnológica, diseño del contrato inteligente o arquitectura del sistema que impacten tiempos/recursos.
- **Resultados de pruebas que obligan a replanificar:** defectos críticos descubiertos que requieran iteraciones adicionales o ajuste de prioridades.

Responsabilidades

- **PM (Andrés F. Manosalva):** gestiona la línea base del plan, coordina las revisiones semanales y mantiene la comunicación con el director.
- **Todos los miembros del equipo:** registran cambios relevantes en el historial de versiones del documento y comunican impactos sobre el plan.
- **Director (Rafael V. Páez):** valida y aprueba los cambios de alcance mayores propuestos por el equipo durante las revisiones semanales.

Control de versiones

- El SPMP se mantiene bajo control de versiones (Git/GitHub).
- El **Historial de Cambios** al inicio del documento registra: versión, fecha, descripción breve del cambio y autor(es).
- Cambios menores (correcciones tipográficas, formato) no requieren nueva versión formal; cambios sustantivos sí.

0.6. Contexto del proyecto

Esta sección describe el contexto en el que se ejecutó el proyecto **SecureTicket**: el modelo de ciclo de vida adoptado, los lenguajes y herramientas utilizados, el plan de aceptación del producto y la estructura organizacional del equipo.

0.6.1. Modelo de ciclo de vida

El equipo adoptó un enfoque **ágil con tablero Kanban** sobre un ciclo de vida **incremental e iterativo**. La elección responde a tres factores específicos del proyecto:

- **Dedicación parcial** de los cuatro integrantes (proyecto académico paralelo a otras asignaturas), incompatible con *sprints* de duración fija.
- **Alta incertidumbre técnica** en la integración con Soroban (SDK aún en evolución, comportamientos no documentados de la *testnet*, ajuste de *trustlines* y *salt* en el despliegue), que exigía flujo continuo y respuesta rápida ante bloqueos.
- **Necesidad de iterar artefactos académicos** (SRS, SDD, SPMP) en paralelo con el código, sin congelar el alcance en fases rígidas.

0.6.1.1. Fases del ciclo de vida ejecutadas

Aunque el trabajo diario se gestionó con flujo continuo, se identificaron **ocho fases incrementales** que estructuraron el proyecto en el semestre 2026-1 (ver Tabla 3). Las

fases se solaparon parcialmente: por ejemplo, el desarrollo del backend (semanas 7–11) comenzó antes de finalizar la implementación de contratos (semanas 6–9).

0.6.1.2. Prácticas Kanban aplicadas

- **Tablero Kanban** en GitHub Projects, con columnas: *Backlog* → *To Do* → *In Progress* → *In Review* → *Done*.
- **Límite WIP** de 2 tareas activas por integrante para evitar dispersión y favorecer cierres efectivos.
- **Definition of Done (DoD)**: criterios mínimos para cerrar una tarea: código en *main* mediante *pull request*, pruebas pasando, revisión por al menos un compañero distinto al autor, y actualización de documentación cuando aplique.
- **Ceremonias ligeras**:
 - *Stand-up* asíncrono diario por canal de mensajería (3 preguntas: qué hice ayer, qué haré hoy, qué me bloquea).
 - Revisión semanal con el director (Rafael V. Pérez), de 45–60 min, donde se revisaban avances, riesgos y artefactos.
 - Revisión interna del equipo al cierre de cada fase incremental, para consolidar lecciones aprendidas.
- **Integración continua manual**: compilación y ejecución de pruebas antes de cada *merge a main*; despliegue manual a *testnet* cuando aplicaba.

0.6.2. Métodos, lenguajes y herramientas

0.6.2.1. Lenguajes de programación

- **Rust** (con `soroban-sdk 23`): contratos inteligentes en Soroban; compilación a WebAssembly con `cargo build -release -target wasm32v1-none`.
- **TypeScript** (Node 20): backend con Express + Prisma, indexer y *scripts* de despliegue.
- **TypeScript / JavaScript** (React 18 + Vite): frontend SPA.
- **SQL** (PostgreSQL 15 en Supabase): consultas analíticas y migraciones gestionadas con Prisma.

0.6.2.2. Herramientas técnicas

La Tabla 6 consolida las herramientas técnicas utilizadas, agrupadas por categoría.

Cuadro 6: Herramientas técnicas del proyecto

Categoría	Herramienta	Uso en el proyecto
Contratos	Soroban SDK 23 (Rust)	Implementación de <code>event_contract</code> , <code>factory_contract</code> y <code>ticket_nft_contract</code> .
Contratos	Stellar CLI / soroban-cli	Compilación, despliegue e invocación en <i>testnet</i> .
Backend	Node.js 20 + Express	API REST.
Backend	Prisma ORM	Modelado de datos, migraciones y <i>type-safety</i> contra PostgreSQL.
Backend	Stellar SDK (JS)	Construcción de XDR, firma y envío de transacciones.
Frontend	React 18 + Vite	SPA y <i>tooling</i> de desarrollo.
Frontend	Freighter Wallet	Firma de transacciones no-custodial por parte del usuario.
Frontend	html5-qrcode	Escáner QR para verificación de boletos en puerta.
Datos	PostgreSQL 15 (Supabase)	Base de datos transaccional <i>off-chain</i> .
Blockchain	Stellar <i>testnet</i>	Red de pruebas para contratos y transacciones.
Blockchain	Soroban RPC + Horizon	Endpoints consumidos por backend e indexer.
Despliegue	Vercel	Hosting del frontend.
Despliegue	Railway / Render	Hosting del backend y del indexer.
VCS	Git + GitHub	Control de versiones, <i>pull requests</i> y revisiones de código.
Gestión	GitHub Projects (Kanban)	Tablero de tareas, asignación y seguimiento WIP.
Comunicación	WhatsApp + Discord	Mensajería diaria y canales temáticos por subsistema.
Reuniones	Microsoft Teams / Google Meet	Revisiones semanales con el director y reuniones internas.
Documentación	L ^A T _E X (MiKTeX, TeXstudio)	Memoria de TG, Plan de Proyecto, SRS, SDD, Propuesta y Formato de Biblioteca.
Diagramas	draw.io / PlantUML	Diagramas C4, BPMN y de secuencia.

0.6.2.3. Métodos y técnicas

- **Modelo C4** (Simon Brown) para representar la arquitectura en niveles de contexto, contenedores y despliegue (ver SDD).
- **Test-driven mindset** en contratos: cada función pública de `event_contract` cuenta con al menos un caso de prueba unitario que cubre la ruta positiva y los errores tipados.
- **Trunk-based development** con *feature branches* de corta duración y *pull requests* hacia `main`.
- **Revisión por pares** obligatoria antes de *merge*.
- **Estimación relativa** con *T-shirt sizing* (S/M/L/XL) en lugar de *story points* numéricos, dada la naturaleza académica del proyecto.

0.6.3. Plan de aceptación del producto

El producto entregado se valida formalmente contra los **criterios de aceptación** definidos en el SRS y refinados en cada revisión semanal con el director. El plan de aceptación distingue tres niveles de validación.

0.6.3.1. Niveles de validación

1. **Validación técnica (interna del equipo):** ejecución exitosa de las suites de pruebas (unitarias en Rust, integración en Node y E2E desde el frontend) sobre la rama `main`, sin defectos críticos abiertos al cierre de la fase.
2. **Validación funcional (revisión con el director):** demostración semanal del estado del flujo E2E sobre la *testnet* (compra, asegurar en blockchain, reventa P2P, verificación en puerta), validando que los criterios de aceptación del SRS se cumplen tal como están escritos.
3. **Aceptación final (entrega y sustentación):** entrega del paquete final de seis documentos (Memoria, Plan de Proyecto, SRS, SDD, Propuesta y Formato de Biblioteca) a la coordinación académica el **24 de mayo de 2026**, y posterior sustentación ante el director y los jurados del trabajo de grado en una fecha asignada por la coordinación dentro de la ventana **25 de mayo – 5 de junio de 2026**, con el sistema desplegado en *testnet*.

0.6.3.2. Criterios de aceptación globales

Para considerar el producto aceptado se requirió el cumplimiento de los siguientes criterios:

- **Suite de contratos desplegada:** `event_contract`, `factory_contract` y `ticket_nft_contract` desplegados en *testnet* con direcciones verificables y al menos un evento operativo.
- **Flujo E2E completo:** un usuario autenticado puede navegar el catálogo, comprar un boleto (Web2), asegurarlo on-chain (Web3), revenderlo a otro usuario (*burn/remint* con comisiones) y un verificador autorizado puede redimirlo en puerta (escáner QR).
- **Cobertura de pruebas mínima:** todas las funciones públicas de los contratos tienen al menos un caso de prueba; el backend cubre las rutas críticas con pruebas de integración.
- **Documentación completa:** los seis documentos académicos entregables se encuentran finalizados, revisados y consistentes entre sí.
- **Sin defectos críticos abiertos:** cualquier defecto que impida la ejecución del flujo E2E está corregido al cierre del proyecto.

0.6.3.3. Procedimiento de aceptación

1. Al cierre de cada fase incremental, el equipo ejecuta la suite de pruebas y registra evidencias.
2. En la revisión semanal con el director, se demuestra el avance sobre la *testnet*; el director acepta o solicita correcciones.
3. El paquete final de documentos se envía con al menos una semana de antelación a la sustentación, para revisión previa del director.
4. En la sustentación, los jurados validan funcionalidad y documentación; el equipo atiende observaciones y aplica correcciones menores si aplica.

0.6.4. Estructura organizacional

0.6.4.1. Interfaces externas

El proyecto se relaciona con los siguientes actores externos al equipo:

- **Director de trabajo de grado:** Rafael Vicente Páez Méndez, PhD. Aprueba alcance, valida avances semanalmente y firma los entregables académicos.
- **Jurados del trabajo de grado:** docentes asignados por la Universidad para la sustentación final.
- **Coordinación académica** de Ingeniería de Sistemas, Pontificia Universidad Javeriana Bogotá: define el calendario académico oficial y los formatos institucionales.
- **Comunidad Stellar / Soroban:** fuente de documentación técnica, foros de soporte y actualizaciones del SDK.

0.6.4.2. Estructura interna del equipo

El equipo (Grupo 12) se organiza como una célula **auto-gestionada de cuatro integrantes** bajo coordinación de un PM, sin jerarquías formales. Las decisiones técnicas y de alcance se toman por consenso en las reuniones semanales internas; el PM resuelve empates cuando es necesario y representa al equipo ante el director.

La Tabla 7 resume los roles y responsabilidades principales por integrante.

Cuadro 7: Roles y responsabilidades del equipo

Integrante	Rol principal	Responsabilidades
Andrés Felipe Manosalva Garzón	PM / Coordinación	Coordinación del equipo, planificación, seguimiento del tablero Kanban, comunicación con el director, mantenimiento del SPMP y apoyo transversal en frontend y documentación.
Santiago Andrés Mesa Niño	Blockchain (Soroban)	Diseño e implementación de los contratos en Rust, pruebas unitarias, despliegue en <i>testnet</i> y mantenimiento de los <i>scripts</i> de <i>deploy</i> .
Juan Diego Arias Durán	Backend	Diseño e implementación de la API Express + Prisma, XDR proxy, indexer Soroban, esquema PostgreSQL y arreglos puntuales en el frontend.
Juan Camilo Carvajal Rodríguez	Frontend QA/DevOps	+ Implementación de la SPA React/Vite, integración Freightier, escáner QR, ejecución de pruebas E2E y despliegues a Vercel/Railway.

0.6.4.3. Matriz de responsabilidades (RACI resumida)

Una **matriz RACI** es una herramienta clásica de gestión de proyectos que clarifica, para cada entregable, el grado de involucramiento de cada persona u rol. Las siglas **R**, **A**, **C**, **I** corresponden a cuatro tipos distintos de responsabilidad, descritos a continuación:

- **R — Responsable (Responsable de ejecutar)**. Persona(s) que **realiza(n) el trabajo**. Es quien “mete las manos” en la tarea. Puede haber más de un **R** por entregable cuando el trabajo es compartido.
- **A — Accountable (Aprobador / Rinde cuentas)**. Persona que **aprueba el resultado final** y responde por él ante terceros. Debe existir **exactamente un A** por

entregable para evitar diluir la responsabilidad. En este proyecto, el director del trabajo de grado es siempre el **A** de los entregables académicos.

- **C — Consulted (Consultado).** Persona(s) cuya **opinión técnica se consulta antes de tomar decisiones** sobre el entregable. La consulta es *bidireccional*: hay diálogo. Suelen ser expertos en el subsistema afectado.
- **I — Informed (Informado).** Persona(s) a las que se les **comunica el resultado** una vez tomada la decisión o avanzado el trabajo, pero sin consulta previa. La comunicación es *unidireccional*.

Cómo leer la matriz: cada fila representa un entregable y cada columna una persona. La celda indica qué tipo de involucramiento tiene esa persona con ese entregable. Por ejemplo, en la fila “Contratos Soroban”: **SM** es **R** (Santiago ejecuta el trabajo), **AM** y **JA** son **C** (se consulta a Andrés por temas de planeación y a Juan Diego por la integración con el backend), **JC** es **I** (se le informa del avance) y el **Dir.** es **A** (el director aprueba el resultado final).

Para abreviar se usan iniciales: **AM** (Andrés Manosalva), **SM** (Santiago Mesa), **JA** (Juan Diego Arias), **JC** (Juan Camilo Carvajal) y **Dir.** (Director del trabajo de grado).

Cuadro 8: Matriz RACI por entregable principal

Entregable	AM	SM	JA	JC	Dir.
Contratos Soroban	C	R	C	I	A
Backend + Indexer	C	C	R	I	A
Frontend + Freightier	C	I	C	R	A
Despliegue <i>testnet</i> (12 eventos)	C	C	C	R	A
Plan de Proyecto (SPMP)	R	I	C	I	A
SRS	R	C	C	C	A
SDD	C	C	R	C	A
Memoria de TG	R	C	C	C	A
Propuesta de TG	R	I	I	I	A
Formato de Biblioteca	R	I	I	I	A
Demo final	C	C	C	R	A

0.6.4.4. Canales de comunicación

- **Mensajería diaria:** grupo de WhatsApp del equipo para coordinación rápida y *stand-up* asíncrono.

- **Discord:** canales temáticos por subsistema (*contracts*, *backend*, *frontend*, *docs*) para discusiones técnicas más extensas.
- **GitHub Issues / Pull Requests:** discusión formal asociada al código, decisiones de diseño y revisiones.
- **Microsoft Teams / Google Meet:** reuniones semanales con el director y reuniones de cierre de fase del equipo.
- **Correo institucional:** comunicación formal con coordinación académica y entrega de documentos a la Universidad.

0.7. Administración del proyecto

Esta sección documenta cómo se administró el proyecto durante el semestre 2026-1: los métodos y herramientas de estimación de esfuerzo, el plan de inicio (entrenamiento e infraestructura) y los planes de trabajo detallados (estructura desglosada del trabajo, carta Gantt, asignación de recursos y presupuesto consolidado).

0.7.1. Métodos y herramientas de estimación

0.7.1.1. Enfoque general

Dada la naturaleza académica del proyecto y la dedicación parcial del equipo, no se aplicaron técnicas formales de estimación paramétrica (COCOMO, Function Points). En su lugar, se combinaron dos técnicas ágiles ampliamente aceptadas para equipos pequeños:

- **T-shirt sizing** (S/M/L/XL) para clasificar tareas del *backlog* según el esfuerzo relativo percibido por el integrante con mayor experiencia técnica en el área correspondiente.
- **Estimación por analogía:** comparación con tareas similares ya completadas dentro del proyecto (por ejemplo, estimar el tiempo de una nueva función del contrato a partir de las funciones implementadas y probadas con anterioridad).

La correspondencia aproximada entre tallas y horas-persona se muestra en la Tabla 9. Esta correspondencia se calibró durante las primeras dos fases del proyecto y se mantuvo estable hasta el cierre.

Talla	Horas	Ejemplo típico
S	1–2	Ajuste de estilos en una pantalla del frontend; corrección de un texto del SRS.
M	3–5	Implementación de un <i>endpoint</i> REST con pruebas básicas; un caso de prueba de contrato.
L	6–10	Una función pública nueva del <code>event_contract</code> con su batería de pruebas.
XL	11+	Indexer Soroban; <i>burn/remint</i> atómico; integración completa de Freight.

Cuadro 9: Correspondencia T-shirt sizing $\leftrightarrow$ horas-persona

0.7.1.2. Herramientas de estimación y seguimiento

- **GitHub Projects** (Kanban): cada tarjeta del tablero registra su talla estimada como etiqueta (`size: S/M/L/XL`).
- **Hoja de cálculo de horas** compartida en Google Drive: cada integrante registra semanalmente las horas reales invertidas por tarea, permitiendo calibrar la estimación.
- **Issues de GitHub**: las tareas complejas se descomponen en *checklists* antes de empezar, lo que reduce el sesgo optimista en la estimación.

0.7.1.3. Calibración y aprendizajes

Las primeras tres iteraciones del proyecto presentaron una desviación promedio de +30% entre las horas estimadas y las horas reales, especialmente en tareas relacionadas con Soroban (debido a la curva de aprendizaje del SDK y problemas no documentados con el `target wasm32v1-none`). A partir de la fase de implementación del backend, la desviación bajó a ~10% gracias a la calibración de las tallas y a la práctica acumulada del equipo.

0.7.2. Plan de inicio del proyecto

El plan de inicio define las actividades ejecutadas durante las dos primeras semanas (**semanas 1–2 del semestre 2026-1**) para habilitar al equipo y la infraestructura antes de empezar el desarrollo.

0.7.2.1. Entrenamiento y nivelación

Las áreas técnicas con mayor incertidumbre exigieron una fase explícita de capacitación:

- **Stellar y Soroban:** lectura de la documentación oficial (*Stellar Documentation*, *Soroban Smart Contract Reference*), realización del *tutorial* oficial “Hello World” de Soroban y revisión de los *contracts examples* del repositorio público de Stellar. Responsable: Santiago Mesa (Blockchain); apoyo de todo el equipo.
- **Freighter Wallet:** estudio del SDK *Freighter API* y pruebas locales de firma de transacciones. Responsable: Juan Camilo Carvajal (Frontend).
- **Prisma + Supabase:** revisión de la documentación de Prisma 5, ejecución de migraciones de ejemplo contra una instancia gratuita de Supabase. Responsable: Juan Diego Arias (Backend).
- **Patrones y metodologías:** repaso interno del modelo C4, del estándar IEEE 1058 (este documento) y del flujo Kanban. Responsable: Andrés Manosalva (PM); difusión a todo el equipo.

0.7.2.2. Infraestructura inicial

Durante las semanas 1–2 también se aprovisionó la infraestructura mínima necesaria para arrancar:

- **Repositorio Git** en GitHub bajo la organización *Tesis-Stellar* con el nombre *Secure-Ticket*; configuración de ramas protegidas (**main**), plantillas de *pull request* y archivo CODEOWNERS.
- **Tablero Kanban** en GitHub Projects con columnas *Backlog* → *To Do* → *In Progress* → *In Review* → *Done* y etiquetas por subsistema (**contracts**, **backend**, **frontend**, **docs**, **infra**).
- **Cuentas y proyectos** creados en: Supabase (PostgreSQL), Vercel (frontend), Railway (backend e indexer); todos en planes *free tier* académicos.
- **Wallets de testnet** generadas para los roles del sistema: **ADMIN**, **ORGANIZER**, **PLATFORM**, **BUYER1**, **BUYER2**, **VERIFIER**; semillas de 12 palabras almacenadas en `backend/.env.deploy` (*gitignored*).
- **Entorno local** unificado: documento `SETUP.md` con instrucciones para Rust + cargo, Node 20, soroban-cli y configuración de Freighter en modo *testnet*.
- **Carpeta compartida en Google Drive** para los borradores LaTeX y materiales de apoyo.

0.7.3. Planes de trabajo

0.7.3.1. Estructura desglosada del trabajo (WBS)

La WBS organiza el trabajo del proyecto en tres niveles jerárquicos: **nivel 1** (proyecto), **nivel 2** (paquetes de trabajo por subsistema o tipo de artefacto) y **nivel 3** (entregables o componentes específicos). El esquema se presenta a continuación.

1.1 Gestión del proyecto

- Plan de Proyecto (SPMP) y sus versiones (v1.0 a v1.5).
- Tablero Kanban y seguimiento semanal.
- Revisiones semanales con el director y actas internas.
- Cierre del proyecto y *post-mortem*.

1.2 Análisis y diseño

- Requisitos funcionales y no funcionales (SRS).
- Modelo de dominio Web2.5 (catálogo, carrito, boleto, NFT, evento, comisiones).
- Vistas C4 (contexto, contenedores, despliegue) y diagramas de secuencia.
- Modelo de datos on-chain (*storage keys*, eventos) y off-chain (Prisma schema).
- SDD (Especificación del Diseño).

1.3 Contratos inteligentes (Soroban / Rust)

- `event_contract`: emisión, listado, compra primaria, reventa *burn/remint*, verificadores, redención, invalidación.
- `factory_contract`: patrón Factory por evento.
- `ticket_nft_contract`: NFT SEP-41 para Freightier Collectibles.
- Pruebas unitarias e integración (Rust `#[test]`).
- Scripts de despliegue y configuración de `testnet`.

1.4 Backend (Express + Prisma + Indexer)

- API REST: catálogo, autenticación (bcrypt+JWT), carrito, *checkout*, órdenes, boletos.
- Constructor de XDR *stateless* (XDR proxy).
- Indexer Soroban (polling cada 5 s) y sincronización a PostgreSQL.
- Esquema Prisma y migraciones versionadas.
- Documentación de endpoints.

1.5 Frontend (React + Vite)

- Catálogo, detalle de evento, carrito y *checkout*.
- Cuenta de usuario, órdenes, boletos.
- Integración Freightier Wallet (conexión, firma, asegurar on-chain).
- Mercado secundario P2P (publicar reventa, comprar reventa, cancelar).
- Escáner QR para verificación en puerta.
- Panel de administración (Phase 4) y vistas para rol STAFF.

1.6 Pruebas y aseguramiento de calidad

- Plan y casos de prueba.
- Ejecución de pruebas unitarias, integración y E2E.
- Evidencias y reporte de defectos.
- Validación contra los criterios de aceptación del SRS.

1.7 Despliegue e infraestructura

- Despliegue de los 12 eventos demostrativos en *testnet*.
- Configuración de Vercel (frontend), Railway (backend, indexer) y Supabase (PostgreSQL).
- Variables de entorno y secretos.

1.8 Documentación académica final

- Memoria de TG.
- Propuesta de TG.
- SRS final.
- SDD final.
- Plan de Proyecto v1.5 (este documento).
- Formato de Biblioteca.

1.9 Demo y sustentación final

- Guion del demo (flujo E2E sobre *testnet*).
- Grabación opcional en video del recorrido.
- Sustentación ante director y jurados (ventana 25 de mayo – 5 de junio de 2026, fecha exacta por confirmar).

0.7.3.2. Carta Gantt ejecutada

La Tabla 10 representa la carta Gantt del proyecto a nivel de paquete WBS y semana del semestre. Las celdas marcadas con un punto (●) indican semanas de actividad principal del paquete; el cierre y la entrega documental son el viernes **24 de mayo de 2026** (semana 16). La sustentación se programa en una fecha asignada por la coordinación dentro de la ventana **25 de mayo – 5 de junio de 2026** y, por estar fuera del semestre académico, no se grafica en la carta Gantt.

Cuadro 10: Carta Gantt ejecutada (semanas 1–16 del semestre 2026-1)

Paquete WBS	Semana															
	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1. Gestión	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●	●
2. Análisis y diseño	●	●	●	●	●											
3. Contratos						●	●	●	●							
4. Backend + Indexer							●	●	●	●	●					
5. Frontend									●	●	●	●				
6. Pruebas/QA														●	●	
7. Despliegue												●	●	●	●	
8. Documentación final															●	●
9. Demo / sustentación																●

0.7.3.3. Asignación de recursos

La asignación de recursos humanos al esfuerzo de cada paquete WBS se resume en la Tabla 11. La columna *Líder* indica el integrante responsable del paquete; las demás columnas indican el porcentaje aproximado del esfuerzo total del integrante dedicado a ese paquete.

Cuadro 11: Asignación de recursos por paquete WBS (% del esfuerzo del integrante)

Paquete WBS	Líder	AM	SM	JA	JC
1. Gestión	AM	35 %	5 %	5 %	5 %
2. Análisis y diseño	AM / JA	20 %	10 %	20 %	10 %
3. Contratos Soroban	SM	5 %	55 %	10 %	5 %
4. Backend + Indexer	JA	5 %	10 %	40 %	10 %

Paquete WBS	Líder	AM	SM	JA	JC
5. Frontend	JC	5 %	5 %	10 %	35 %
6. Pruebas y QA	JC	10 %	10 %	5 %	20 %
7. Despliegue	JC	5 %	5 %	5 %	10 %
8. Documentación final	AM	10 %	—	5 %	5 %
9. Demo / sustentación	JC	5 %	—	—	—
Total		100 %	100 %	100 %	100 %

Convertido a horas-persona (a partir de la Tabla 4: AM 60 h, SM 65 h, JA 65 h, JC 65 h), el esfuerzo total invertido por paquete se muestra en la Tabla 12.

Paquete WBS	Horas totales
1. Gestión	30
2. Análisis y diseño	32
3. Contratos Soroban	45
4. Backend + Indexer	38
5. Frontend	32
6. Pruebas y QA	28
7. Despliegue	16
8. Documentación final	16
9. Demo / sustentación	3
Indirecto / reuniones / otros	15
Total	255

Cuadro 12: Horas-persona ejecutadas por paquete WBS

0.7.3.4. Presupuesto detallado

El presupuesto monetario consolidado del proyecto se presentó en la Tabla 5 de la Sección 0.5.5 (total **748.000 COP**, con contingencia del 10 %). La Tabla 13 desagrega el gasto por fase del proyecto, vinculando los rubros operativos con el momento en que se incurrieron.

Fase / Momento	Rubro	Monto (COP)
Semanas 1–16 (semestre)	Servicios de luz (4 hogares)	480,000
Semanas 3–13 (revisiones)	Transporte / viáticos	60,000
Semanas 15–16 (cierre)	Transporte / viáticos (sustentación)	20,000
Semana 16 (entrega)	Impresiones de anexos	120,000
Subtotal operativo		680,000
Reserva (contingencia 10 %)	Imprevistos	68,000
Total		748,000

Cuadro 13: Presupuesto monetario desagregado por fase y rubro

Observaciones de presupuesto:

- El presupuesto se ejecutó **íntegramente dentro de lo planificado**, sin necesidad de activar la contingencia.
- Los servicios de luz se asumen como costo indirecto del trabajo desde casa, prorrateado por integrante.
- Los costos de hosting (Vercel, Railway, Supabase) y de licencias de herramientas (GitHub, Freightier, Stellar testnet) fueron **0 COP**, gracias a planes gratuitos.
- El equipo financió íntegramente los gastos operativos como aporte propio al trabajo de grado.

0.8. Monitoreo y control del proyecto

Esta sección documenta los mecanismos que el equipo utilizó para **vigilar el avance** del proyecto, **controlar** las desviaciones respecto al plan y **cerrar formalmente** las fases incrementales. Cubre tres bloques: la administración de requerimientos a lo largo del semestre, el monitoreo de progreso mediante métricas y KPIs, y el proceso de cierre de iteraciones.

0.8.1. Administración de requerimientos

0.8.1.1. Origen y trazabilidad de los requerimientos

Los requerimientos del sistema se documentaron formalmente en el **SRS**. Cada requerimiento tiene un identificador único (**RF-XX** para funcionales, **RNF-XX** para no funcionales) que se referencia en:

- Los *issues* de GitHub que implementan la funcionalidad.

- Los casos de prueba que la validan.
- Los diagramas del SDD que la describen.
- Las acciones de aceptación con el director.

Esta trazabilidad bidireccional permitió, en cualquier momento del proyecto, responder a preguntas como “¿qué pruebas cubren RF-07?” o “¿qué requerimiento justifica esta función del contrato?”.

0.8.1.2. Proceso de cambio de requerimientos

Durante el semestre se produjeron varios cambios al alcance original; los más relevantes están registrados en el historial de cambios del SRS y del SPMP. El procedimiento aplicado para cada cambio fue:

1. **Detección:** cualquier integrante o el director identifica un requerimiento ambiguo, faltante o desalineado con la realidad técnica.
2. **Propuesta:** se abre un *issue* en GitHub etiquetado como **scope-change**, con descripción del cambio y motivación.
3. **Análisis de impacto:** el PM (Andrés Manosalva) evalúa el impacto en alcance, calendario y presupuesto. Si el impacto es mayor, se consulta al subgrupo técnico afectado.
4. **Aprobación:** en la revisión semanal con el director se discute y se aprueba o rechaza el cambio.
5. **Registro:** el cambio se documenta en el historial de cambios del SRS, y si afecta planeación, también en el SPMP.
6. **Comunicación:** se actualiza el tablero Kanban y se informa al equipo en el canal de Discord correspondiente.

0.8.1.3. Cambios de alcance materializados

Los cambios de alcance más relevantes durante el semestre 2026-1 se resumen en la Tabla [14](#).

Cuadro 14: Cambios de alcance aprobados durante el semestre

Fecha	Cambio	Motivo / Impacto
Mar. 2026	Reemplazo de NestJS por Express en el backend	Curva de aprendizaje de NestJS demasiado alta para el tiempo disponible; Express ofrecía la flexibilidad necesaria con menor sobrecarga. Sin impacto en alcance funcional.
Abr. 2026	Eliminación del modo de verificación <i>offline</i> en puerta	Complejidad de implementación (sincronización al volver <i>online</i>) no justificada en alcance académico. Liberó ~15 h-persona reasignadas a Phase 4.
Abr. 2026	Incorporación de Phase 4 (panel admin + STAFF)	Necesidad detectada durante pruebas de integración: organizadores requerían gestión visual de eventos y los verificadores necesitaban un rol diferenciado. Aprobado por el director.
Abr. 2026	Adopción de <code>ticket_nft_contract</code> (SEP-41)	Freighter clasificaba los Classic Assets como “Tokens” en lugar de “Collectibles”; SEP-41 corrige la presentación. Añadió un contrato nuevo al alcance.
May. 2026	Eliminación de Plan de Pruebas, Guía de Despliegue, Manual de Usuario y Póster como entregables formales	Reorganización del paquete documental final a los seis documentos institucionales (Memoria, Plan de Proyecto, SRS, SDD, Propuesta, Formato de Biblioteca). La información se consolidó en la Memoria de TG.

0.8.2. Monitoreo y control de progreso

0.8.2.1. Ciclo de monitoreo

El monitoreo siguió un **ciclo semanal sincrónico** respaldado por una **observación diaria asíncrona**:

- **Diario (asíncrono)**: cada integrante actualiza el estado de sus tareas en el tablero

Kanban antes de las 22:00 y envía un breve mensaje en el canal de WhatsApp con avances y bloqueos.

- **Semanal (sincrónico):** reunión de 45–60 min con el director donde se revisan métricas, riesgos y entregables; reunión interna del equipo de 30 min para planear la semana siguiente.
- **Fin de fase:** sesión de cierre donde se valida el cumplimiento de los objetivos de la fase y se levantan lecciones aprendidas.

0.8.2.2. KPIs definidos y resultados

Se definieron **siete KPIs** principales para monitorear avance, calidad y salud del proyecto. La Tabla 15 consolida el indicador, su umbral objetivo y el valor real al cierre del semestre.

Cuadro 15: KPIs del proyecto: definición, umbral y resultado final

KPI	Fórmula	Umbral	Resultado
% de pruebas aprobadas (contratos)	pruebas OK / total	$\geq 95 \%$	100 % (58/58)
% de cobertura de funciones públicas con pruebas	funciones cubiertas / públicas	$\geq 90 \%$	100 %
Eventos desplegados en <i>test-net</i>	eventos desplegados / planificados	$\geq 80 \%$	12/12 (100 %)
Defectos críticos abiertos al cierre	conteo absoluto	0	0
Hitos cumplidos en fecha	hitos a tiempo / total	$\geq 80 \%$	8/9 (89 %)
Cumplimiento de revisiones semanales con director	realizadas / planificadas	$\geq 85 \%$	15/16 (94 %)
Desviación de horas-persona (real vs. estimado)	$ \text{real} - \text{est.} / \text{est.}$	$\leq 20 \%$	$\sim 10 \%$

Notas sobre los KPIs:

- Las 58 pruebas corresponden a 35 de `event_contract` + 13 de `factory_contract` + 10 de `ticket_nft_contract`.
- El único hito que se desplazó fue “Backend + Indexer” (planeado sem. 7–10, ejecutado sem. 7–11 por la curva de aprendizaje con Prisma + Stellar SDK), lo que no afectó hitos posteriores.

- La revisión semanal pendiente correspondió a la semana de receso académico de Semana Santa.

0.8.2.3. Métricas auxiliares de seguimiento

Además de los KPIs principales, se monitorearon métricas operativas obtenidas del repositorio Git y del tablero Kanban:

- **Lead time promedio** de una tarjeta Kanban (desde *In Progress* hasta *Done*): ~ 3 días.
- **Cycle time promedio** de un *pull request* (apertura $\rightarrow$ merge): $\sim 1,5$ días.
- **Tasa de *pull requests* con observaciones** en revisión: $\sim 60\%$, lo que indica que la revisión por pares fue efectiva (no era un “rubber stamp”).
- **Frecuencia de *commits* en main**: pico en semanas 7–12 (desarrollo intensivo), caída controlada en semanas 15–16 (consolidación documental).

0.8.2.4. Acciones correctivas aplicadas

Cuando las métricas o los riesgos indicaban una desviación, se aplicaron acciones correctivas durante el semestre. Las más relevantes:

- **Semana 4**: la curva de aprendizaje de Soroban era más alta de lo estimado. **Acción**: se programó una sesión de *pair programming* entre Santiago y Andrés para acelerar el dominio del SDK y se ajustaron las estimaciones del paquete WBS de contratos.
- **Semana 7**: el *target wasm32-unknown-unknown* producía WASMs rechazados por la red. **Acción**: migración a *wasm32v1-none*, documentación interna del cambio y actualización del *script* de *build*.
- **Semana 11**: aparecieron desviaciones en el indexer (eventos on-chain no se reflejaban a tiempo en PostgreSQL). **Acción**: se redujo el intervalo de *polling* de 10 s a 5 s y se añadió manejo de reintentos.
- **Semana 13**: riesgo de Phase 4 (panel admin) consumiendo tiempo de documentación. **Acción**: reasignación temporal de Juan Diego al frontend de Phase 4 y *freeze* del alcance del panel.
- **Semana 15**: densidad de documentación pendiente. **Acción**: reuniones diarias breves del equipo (15 min) hasta la entrega final para coordinar cierre de SRS, SDD, Memoria y SPMP en paralelo.

0.8.3. Cierre de fases e iteraciones

0.8.3.1. Criterios de cierre

Para considerar una fase incremental como **cerrada** se debían cumplir simultáneamente los siguientes criterios:

1. Todos los entregables planificados para la fase están *merged* en *main* y desplegados (cuando aplica).
2. Las pruebas asociadas pasan al 100 %.
3. No quedan defectos críticos abiertos asociados al alcance de la fase.
4. El director ha validado el avance en la revisión semanal correspondiente.
5. Los artefactos académicos afectados (SRS, SDD, SPMP) están actualizados.

0.8.3.2. Ritual de cierre de fase

Al cumplir los criterios anteriores, el equipo ejecutaba un ritual de cierre breve (30–45 min) compuesto por cuatro pasos:

1. **Demostración:** el líder de la fase muestra el avance funcional a todo el equipo.
2. **Revisión de métricas:** se actualizan los KPIs de la Sección 0.8.2 y se contrasta con los umbrales.
3. **Lecciones aprendidas:** cada integrante aporta un “qué funcionó” y un “qué mejorar”.
4. **Planeación de la siguiente fase:** el PM ajusta el tablero Kanban y reorganiza prioridades según los aprendizajes.

0.8.3.3. Lecciones aprendidas consolidadas

Las lecciones aprendidas más relevantes, extraídas de los cierres de fase y consolidadas para el cierre final del proyecto, son:

- **Capacitarse antes de planear con detalle.** Las primeras estimaciones sobre Soroban estuvieron infladas por desconocimiento; haber dedicado las semanas 1–2 a entrenamiento ahorró tiempo en fases posteriores.
- **La documentación oficial no siempre coincide con el comportamiento real.** Varios problemas (*target* WASM, reference-types, deploy con *salt*) sólo aparecieron al hacer el primer despliegue real.
- **El indexer es el “pegamento” entre Web2 y Web3** y merece la misma atención de pruebas que los contratos.

- **Freezing del alcance al final.** Decidir *freeze* Phase 4 en semana 13 y dedicar las últimas semanas a estabilizar y documentar fue una decisión acertada.
- **Las revisiones semanales con el director fueron el mejor mecanismo de control.** Mantener la cadencia (incluso cuando el avance era modesto) evitó sorpresas en la sustentación.
- **Mantener la documentación al día desde el inicio** es más barato que dejarla para el final; aún así, el equipo necesitó una recta final intensa, lo que indica que el balance puede mejorarse en proyectos futuros.

0.8.3.4. Cierre formal del proyecto

El cierre formal del proyecto se llevó a cabo en **semana 16**, con tres actividades:

1. Entrega de los seis documentos académicos a la coordinación, en formato PDF, el **24 de mayo de 2026**.
2. Sustentación pública ante director y jurados en una fecha asignada por la coordinación dentro de la ventana **25 de mayo – 5 de junio de 2026**.
3. Reunión interna de *post-mortem* del equipo: validación de cumplimiento de objetivos, revisión final de métricas, archivo del repositorio y documentación de la versión final v1.5 de este SPMP.

0.9. Entrega del producto

Esta sección describe el **plan de transición** mediante el cual el equipo entrega formalmente el producto y la documentación del trabajo de grado a la Pontificia Universidad Javeriana. Dado que **SecureTicket** es un prototipo académico (no un producto comercial), el plan de transición se centra en la entrega documental, el acceso al código fuente y la presentación del sistema ante el director y los jurados.

0.9.1. Estrategia de entrega

El equipo definió una estrategia de entrega **en una sola fase consolidada**, sin pilotos previos ni despliegues escalonados, por tratarse de un trabajo de grado con una fecha única de entrega documental: el **24 de mayo de 2026**. La sustentación se realiza posteriormente, en una fecha asignada por la coordinación dentro de la ventana **25 de mayo – 5 de junio de 2026** (por confirmar). La estrategia se apoya en tres pilares:

- **Entrega documental** del paquete de seis documentos académicos en formato PDF.

- **Entrega del código fuente** mediante enlaces públicos al repositorio Git que aloja el sistema (contratos, backend y frontend), sin transferir el código como archivo descargable.
- **Demostración funcional** del prototipo desplegado en *testnet* ante el director y los jurados, en vivo o como video grabado del recorrido.

No se contempla una entrega a un cliente comercial ni un despliegue en *mainnet*; el receptor del producto es **la Universidad** (director, jurados y biblioteca institucional).

0.9.2. Artefactos a entregar

La Tabla 16 consolida todos los artefactos del paquete final, su formato y su responsable de entrega.

Cuadro 16: Artefactos del paquete de entrega final

Artefacto	Descripción	Formato	Responsable
Memoria de TG	Documento principal del trabajo de grado	PDF (L ^A T _E X)	Todos
Plan de Proyecto (SPMP v1.5)	Este documento	PDF (L ^A T _E X)	Todos
SRS	Especificación de Requisitos de Software	PDF (L ^A T _E X)	Todos
SDD	Especificación del Diseño (vistas C4, secuencias, datos)	PDF (L ^A T _E X)	Todos
Propuesta de TG	Anteproyecto aprobado	PDF	Todos
Formato de Biblioteca	Formato institucional para entrega del TG	PDF	Todos
Enlace al repositorio Git	Código fuente (contratos, backend, frontend)	URL pública	Todos
Direcciones de contratos en <i>testnet</i>	WASM hashes y direcciones de 12 eventos desplegados	Lista en Memoria de TG	Blockchain (Santiago Mesa)
Demo final	Sistema en <i>testnet</i> en vivo o video grabado	Demo/MP4	Todos

Aclaraciones:

- Sólo los **seis documentos** (Memoria, Plan de Proyecto, SRS, SDD, Propuesta y Formato de Biblioteca) se entregan formalmente a la coordinación académica.
- El **código fuente** no se anexa al paquete documental; sólo se entregan los enlaces públicos al repositorio.
- El **Demo final** no se anexa como artefacto descargable; se presenta el día de la sustentación.

0.9.3. Cronograma de entrega

El cronograma de la entrega final, ejecutado durante las dos últimas semanas del semestre, se resume en la Tabla 17.

Cuadro 17: Cronograma de entrega final

Fecha (2026)	Actividad	Responsable
11 – 15 may	Consolidación final de los seis documentos; revisiones cruzadas entre integrantes	Todos
15 may	Compilación final de PDFs y revisión de consistencia entre documentos	PM
16 may	Despliegue final estable de los 12 eventos en <i>testnet</i> ; verificación E2E del prototipo	QA/DevOps + Blockchain
17 may	Revisión final con el director (validación de paquete documental)	PM + Director
18 – 22 may	Aplicación de observaciones del director; grabación del video del demo (respaldo en caso de fallo durante la sustentación); ensayos de la sustentación	Todos
24 may	Entrega del paquete documental (seis PDFs) a coordinación académica — cierre del proyecto	Todos
25 may – 5 jun	Ventana de sustentación ante director y jurados (fecha exacta asignada por la coordinación)	Todos
Posterior a la sustentación	Atención a observaciones de los jurados y subida de versión final al repositorio institucional	Todos

0.9.4. Capacitación y soporte al receptor

Por la naturaleza académica del proyecto, la capacitación al receptor (jurados y director) se limita a la presentación durante la sustentación y al material documental.

0.9.4.1. Capacitación durante la sustentación

- **Presentación expositiva** (~ 20 min): contexto, problema, propuesta arquitectónica, decisiones clave y resultados.
- **Demostración funcional** (~ 10 min): recorrido E2E del prototipo en *testnet* mostrando los flujos principales (compra Web2, asegurar en blockchain, reventa P2P, verificación en puerta).
- **Sesión de preguntas** (~ 15 min): respuesta a inquietudes técnicas y de gestión del trabajo de grado.

0.9.4.2. Material de apoyo

Los jurados disponen, previo a la sustentación, del **paquete documental completo** (seis documentos) para revisión. Durante la sustentación, el equipo atiende los materiales adicionales que los jurados soliciten en ese momento (por ejemplo, enlaces al repositorio, direcciones de contratos en *testnet* o URLs del prototipo desplegado), sin que el equipo se comprometa de antemano a entregar ninguno de ellos como parte obligatoria del paquete.

0.9.4.3. Soporte post-entrega

Tras la sustentación, el equipo asume el siguiente compromiso de soporte:

- **Atención de observaciones de los jurados:** si los jurados solicitan correcciones o ajustes menores en los documentos, el equipo los aplica dentro del plazo establecido por la coordinación académica.
- **Disponibilidad del prototipo en *testnet*:** el frontend, backend e indexer permanecen desplegados al menos hasta el cierre formal del semestre.
- **Repositorio Git:** permanece público para consulta, sin compromiso de mantenimiento ni evolución.

Importante: el equipo **no asume responsabilidad de mantenimiento, evolución funcional ni operación** del prototipo después del cierre del semestre. La transferencia es un *snapshot* académico, no una entrega operativa.

0.9.5. Criterios de aceptación de la entrega

La entrega se considera **aceptada y cerrada** cuando se cumplen simultáneamente las siguientes condiciones:

1. Los seis documentos académicos están entregados a la coordinación en formato PDF y firmados (cuando aplica) por el director.

2. La sustentación ha sido presentada ante director y jurados en la fecha asignada por la coordinación dentro de la ventana 25 de mayo – 5 de junio de 2026.
3. Los jurados emiten su calificación / acta de sustentación según los lineamientos institucionales.
4. Las observaciones de los jurados (si las hubiera) están atendidas y los documentos finales actualizados son subidos al repositorio institucional de la Universidad.

A partir del cumplimiento de estas condiciones, el proyecto **SecureTicket** se considera formalmente **transferido** a la Universidad como parte del trabajo de grado del Grupo 12.

0.10. Procesos de soporte

Esta sección documenta los **procesos transversales** que apoyaron la ejecución del proyecto: el ambiente de trabajo y las reglas internas del equipo, el análisis y la administración de riesgos, la gestión de configuración y documentación, las métricas del proceso y el control de calidad.

0.10.1. Ambiente de trabajo y reglas del equipo

0.10.1.1. Ambiente de trabajo

El equipo trabajó en una modalidad **predominantemente remota** desde las residencias de cada integrante, con reuniones presenciales puntuales (revisiones con el director, sesiones de *pair programming* en momentos críticos y ensayos de la sustentación). El ambiente técnico unificado se aseguró mediante un archivo **SETUP.md** en el repositorio con las versiones exigidas de Rust, Node, soroban-cli y demás dependencias.

0.10.1.2. Reglas del equipo

Las reglas internas del equipo, acordadas en la primera semana del semestre y mantenidas durante todo el proyecto, son:

- **Respeto y diálogo constructivo:** cualquier discrepancia técnica o de gestión se discute en el canal correspondiente; no se permiten descalificaciones personales.
- **Asistencia a reuniones semanales con el director** como prioridad alta; ausencias sólo por causas justificadas y con aviso previo.
- **Asistencia a reuniones internas del equipo** con la misma prioridad que las académicas.

- **Compromiso con las tareas asignadas:** si un integrante anticipa que no podrá cumplir una entrega, lo comunica con al menos 48 h de antelación para reasignar o ajustar prioridades.
- **Revisión por pares obligatoria** en todo *pull request* antes de *merge* a *main*.
- **No *push* directo a main:** todo cambio pasa por rama y *pull request*.
- **Mantener la documentación al día** con cada cambio funcional relevante.
- **Confidencialidad de credenciales:** secretos en archivos *.env gitignored*; nunca se publican en repositorios o canales abiertos.
- **Decisiones por consenso** en reuniones internas; el PM resuelve empates cuando es necesario.

0.10.2. Análisis y administración de riesgos

0.10.2.1. Proceso de gestión de riesgos

La gestión de riesgos siguió un ciclo continuo de cuatro pasos: **identificación** (en cualquier momento por cualquier integrante o por el director), **análisis** (estimación de probabilidad e impacto), **respuesta** (mitigación, transferencia, aceptación o evitación) y **monitoreo** (revisión semanal del estado de cada riesgo activo).

La probabilidad y el impacto se evaluaron en una escala cualitativa de tres niveles (Baja / Media / Alta), y la exposición se calculó como el producto categórico de ambos. Se utilizó la matriz de la Tabla 18.

Probabilidad \ Impacto	Bajo	Medio	Alto
Alta	Medio	Alto	Crítico
Media	Bajo	Medio	Alto
Baja	Bajo	Bajo	Medio

Cuadro 18: Matriz cualitativa de exposición al riesgo

0.10.2.2. Registro de riesgos del proyecto

La Tabla 19 consolida los **principales riesgos identificados** durante el semestre 2026-1, su evaluación inicial, la estrategia de respuesta y el estado al cierre del proyecto.

Cuadro 19: Registro de riesgos del proyecto

ID	Riesgo	Prob.	Imp.	Exp.	Respuesta aplicada	Estado
R1	Curva de aprendizaje de Soroban más alta de lo estimado	Alta	Alto	Crítico	<i>Pair programming</i> en sem. 4; ajuste de estimaciones; documentación interna del SDK	Materializado / Miti- gado
R2	Inestabilidad de la <i>testnet</i> de Stellar (<i>downtimes</i> , cambios de RPC)	Media	Alto	Alto	Configuración de RPC alternativo; reintentos con <i>backoff</i> en backend e indexer	Materializado / Miti- gado
R3	Cambios en <i>soroban-sdk</i> durante el semestre	Media	Medio	Medio	Fijar versión SDK en <i>Cargo.toml</i> ; documentar incompatibilidades	Materializado / Miti- gado
R4	Pérdida o exposición de llaves privadas de <i>testnet</i>	Baja	Alto	Medio	<i>.env.deploy gitignored</i> ; semillas en gestor local; rotación si se sospecha exposición	No ma- teriali- zado
R5	Desincronización entre estado on-chain y PostgreSQL	Media	Alto	Alto	Indexer con <i>polling</i> cada 5 s; reintentos; mecanismo de reconciliación manual	Materializado / Miti- gado
R6	Indisponibilidad de un integrante por carga académica	Alta	Medio	Alto	Asignación de <i>back-up</i> en cada paquete WBS; reasignación temporal en sem. 13	Materializado / Acep- tado
R7	Cambios de alcance impulsados por revisiones del director	Media	Medio	Medio	Proceso de cambio formal (Sección 0.8.1); evaluación de impacto antes de aceptar	Materializado / Ges- tionado
R8	Defectos críticos descubiertos en la recta final	Media	Alto	Alto	<i>Freeze</i> de alcance en sem. 13; <i>focus</i> de QA en sem. 14–15	No ma- teriali- zado

ID	Riesgo	Prob.	Imp.	Exp.	Respuesta aplicada	Estado
R9	Bloqueo en sustentación por fallo del prototipo en vivo	Media	Alto	Alto	Grabación de video del demo como respaldo	No materializado (mitigado preventivamente)
R10	Inconsistencias entre documentos académicos (SRS, SDD, SPMP, Memoria)	Alta	Medio	Alto	Revisiones cruzadas en sem. 15; trazabilidad por IDs	Materializado / Mitigado
R11	Costos de infraestructura inesperados (cuotas excedidas en <i>free tier</i>)	Baja	Medio	Bajo	Monitoreo de consumo; contingencia presupuestal del 10 %	No materializado
R12	Pérdida de datos por borrado accidental del repositorio o base de datos	Baja	Alto	Medio	<i>Backups</i> automáticos de Supabase; ramas protegidas en GitHub	No materializado

0.10.2.3. Aprendizajes sobre gestión de riesgos

Los riesgos **técnicos relacionados con Soroban** (R1, R2, R3, R5) tuvieron el mayor impacto real y consumieron la mayor parte de las acciones correctivas del proyecto. Los riesgos **operativos** (R6, R7) se gestionaron de forma rutinaria a través de las reuniones semanales. Los riesgos **preventivos para la sustentación** (R8, R9) se monitorearon de cerca en las últimas semanas y se mitigaron preventivamente.

0.10.3. Administración de configuración y documentación

0.10.3.1. Identificación de elementos de configuración

Los **elementos de configuración (CI)** del proyecto, sometidos a control formal de versiones, son:

- **Código fuente:** contratos Rust, backend TypeScript, frontend TypeScript, esquemas Prisma, *scripts* de despliegue.

- **Documentos académicos:** fuentes L^AT_EX de Memoria de TG, SPMP, SRS, SDD, Propuesta y Formato de Biblioteca, junto con sus imágenes/diagramas.
- **Configuración de infraestructura:** `.env.example`, `Cargo.toml`, `package.json`, archivos de Prisma, archivos de despliegue para Vercel y Railway.
- **Artefactos compilados:** archivos WASM de los contratos y sus *hashes* (no versionados en Git, pero registrados en la documentación).

0.10.3.2. Control de versiones

- **Repositorio único** en GitHub bajo la organización *Tesis-Stellar*, con el código y los documentos L^AT_EX en carpetas separadas.
- **Rama main protegida:** requiere *pull request* aprobado, revisión por pares y CI verde antes de *merge*.
- **Ramas de trabajo** con prefijos por tipo: `feat/`, `fix/`, `docs/`, `chore/`.
- **Convención de *commits*** basada en *Conventional Commits* (tipo: descripción) para facilitar el rastreo del historial.
- ***Tags* de versión** para hitos significativos del código (no obligatorios).

0.10.3.3. Control de cambios

Todo cambio sobre un elemento de configuración sigue el flujo:

1. Creación de *issue* (cuando aplica) con descripción del cambio.
2. Apertura de rama y desarrollo del cambio.
3. *Pull request* con descripción, vinculación a *issue* y, cuando aplica, evidencias visuales (capturas, GIFs).
4. Revisión por al menos un compañero distinto al autor.
5. Ajustes solicitados (si los hay) y aprobación.
6. *Merge* a `main` mediante *squash* o *merge commit* estándar (según política por carpeta).

0.10.3.4. Gestión de versiones de los documentos

- Cada documento académico mantiene su propio **historial de cambios** al inicio del documento, con fecha, descripción y autor.
- Los documentos se compilan a PDF localmente con `pdflatex`; el PDF compilado más reciente se versiona en el repositorio cuando hay una revisión sustancial.

- Las versiones de los documentos se sincronizan con los hitos del proyecto (por ejemplo, este SPMP llega a v1.5 al cierre).

0.10.3.5. Respaldos

- Repositorio replicado en GitHub (servicio externo, redundancia automática).
- Supabase realiza *backups* automáticos diarios del esquema PostgreSQL.
- Cada integrante mantiene una copia local del repositorio actualizada.

0.10.4. Métricas y proceso de medición

Esta subsección describe **cómo** se midió el proyecto. Los **valores finales** de los KPIs se reportaron en la Sección [0.8.2](#).

0.10.4.1. Proceso de medición

1. **Definición:** cada KPI cuenta con un nombre, fórmula, umbral y fuente de datos definidos antes de empezar a medirlo.
2. **Recolección:** las métricas se recolectan al final de cada semana, con responsable claro (PM para métricas de gestión, líder de subsistema para métricas técnicas).
3. **Almacenamiento:** se mantienen en una hoja compartida en Google Drive con histórico por semana.
4. **Análisis:** se revisan en la reunión semanal con el director; las desviaciones se discuten y, si aplica, se activan acciones correctivas.
5. **Reporte:** la versión consolidada se incorpora a este SPMP y a la Memoria de TG.

0.10.4.2. Categorías de métricas

- **Métricas de progreso:** hitos cumplidos, eventos desplegados, cumplimiento de cronograma.
- **Métricas de calidad:** % de pruebas aprobadas, cobertura, defectos abiertos, defectos críticos.
- **Métricas de proceso:** *lead time*, *cycle time* de *pull requests*, tasa de *PRs* con observaciones.
- **Métricas de esfuerzo:** desviación de horas-persona estimadas vs. reales.
- **Métricas de colaboración:** asistencia a reuniones semanales, frecuencia de *commits* por integrante.

0.10.4.3. Fuentes de datos

- **GitHub:** *commits*, *pull requests*, *issues*, tablero Kanban (GitHub Projects).
- **Suites de prueba:** salida de `cargo test` (contratos), `npm test` (backend) y pruebas E2E del frontend.
- ***Block explorer* de Stellar:** verificación del estado de contratos desplegados.
- **Hoja de horas** compartida en Google Drive.

0.10.5. Control de calidad

El control de calidad se apoya en tres mecanismos complementarios: **verificación** (¿lo estamos construyendo bien?), **validación** (¿estamos construyendo lo correcto?) y **revisiones** (inspección humana).

0.10.5.1. Verificación

- **Pruebas unitarias** en cada función pública de los contratos (Rust `#[test]`).
- **Pruebas de integración** en el backend, cubriendo rutas críticas (autenticación, *checkout*, operaciones on-chain).
- **Pruebas E2E** desde el frontend cubriendo el flujo: compra Web2 → asegurar on-chain → reventa P2P → verificación en puerta.
- **Análisis estático:** `cargo clippy` y `cargo fmt` en Rust; `tsc -noEmit` y `eslint` en TypeScript.
- **Verificación de *builds*:** compilación local exitosa antes de cada *pull request*.

0.10.5.2. Validación

- **Validación con el director:** demostración semanal del avance E2E sobre la *testnet*.
- **Validación contra criterios de aceptación del SRS:** antes del cierre de cada fase incremental.
- **Validación cruzada del equipo:** cada integrante valida los flujos cuya implementación no le pertenece, para reducir sesgos del autor.
- **Validación final:** ejecución del flujo E2E completo en *testnet* con los 12 eventos desplegados.

0.10.5.3. Revisiones e inspecciones

- **Revisión por pares (*code review*)** en todo *pull request*: al menos un revisor distinto al autor.
- **Revisión de documentos**: revisiones cruzadas entre integrantes antes de marcar un documento como final.
- **Revisión con el director**: validación semanal del estado de los artefactos académicos y del prototipo.
- **Inspección de configuración**: verificación periódica de que no haya secretos en el repositorio público.

0.10.5.4. Definition of Done (DoD)

Una tarea o entregable se considera **hecho** cuando cumple, según aplique:

- Código en *main* mediante *pull request* aprobado.
- Pruebas asociadas al cambio pasando al 100 %.
- Documentación actualizada (si la funcionalidad lo amerita).
- No introduce regresiones en los flujos E2E ya estables.
- Para documentos académicos: revisión cruzada interna y, si aplica, validación con el director.

0.10.5.5. Resumen de resultados de calidad

Al cierre del proyecto, los principales indicadores de calidad son:

- **58/58 pruebas** de contratos aprobadas (Sección 0.8.2).
- **100 %** de funciones públicas de los contratos cubiertas por al menos una prueba.
- **12/12 eventos** desplegados y verificables en *testnet*.
- **0 defectos críticos** abiertos.
- **Seis documentos académicos** revisados internamente y validados con el director.

0.11. Cierre del proyecto

Esta sección consolida, de forma retrospectiva, el balance final del trabajo de grado al 24 de mayo de 2026. Recoge el cumplimiento de los compromisos, las lecciones aprendidas transversales, los riesgos residuales y las recomendaciones para una eventual continuación de SecureTicket.

0.11.1. Balance final del proyecto

Al cierre del semestre 2026-1, SecureTicket se entrega como un **prototipo Web2.5 funcional desplegado en *testnet*** de Stellar, con los tres contratos Soroban operativos (`event_contract`, `factory_contract`, `ticket_nft_contract`), 12 eventos desplegados y verificables *on-chain*, un backend Node/Express con indexer activo, un frontend React/Vite integrado con Freightier Wallet y un panel de administración (Phase 4) con roles diferenciados (ADMIN, ORGANIZADOR, STAFF, USER).

- **Alcance entregado:** 100 % de los objetivos comprometidos en la versión 1.0 del SPMP, más la incorporación de la **Phase 4** (panel administrativo y rol STAFF), no contemplada inicialmente.
- **Calidad técnica:** 58/58 pruebas de contratos aprobadas, 0 defectos críticos abiertos al cierre, cobertura del 100 % de funciones públicas de los contratos.
- **Documentación académica:** seis documentos entregados y validados con el director (Memoria de TG, Plan de Proyecto, SRS, SDD, Propuesta de TG y Formato de Biblioteca).
- **Cronograma:** 8/9 hitos cumplidos en fecha; el único desplazamiento (Backend + Indexer) se absorbió sin afectar la fecha de cierre.
- **Presupuesto:** ejecutado dentro del marco operativo planificado (servicios públicos del equipo, sin costos de infraestructura por uso de planes gratuitos de Supabase, Vercel y Railway y de la *testnet* de Stellar).

0.11.2. Cumplimiento de objetivos

La Tabla 20 resume el grado de cumplimiento de los objetivos comprometidos en la Sección 0.5.

Cuadro 20: Cumplimiento de objetivos del proyecto

Objetivo	Estado	Evidencia
Diseñar una arquitectura Web2.5 que combine experiencia Web2 tradicional con NFT opcional.	Cumplido	SDD, vistas C4 y demo E2E.
Implementar tres contratos Soroban (evento, factoría, NFT SEP-41) con pruebas unitarias.	Cumplido	58/58 pruebas; código en <code>main</code> .
Construir un backend con patrón XDR proxy <i>stateless</i> (sin custodia de llaves).	Cumplido	Backend Express + Prisma; auditoría de no-custodia.

Objetivo	Estado	Evidencia
Construir un frontend integrado con Freightier Wallet y catálogo Web2.	Cumplido	Frontend React/Vite desplegado en Vercel.
Implementar reventa atómica P2P (<i>burn/remint</i>) con comisiones organizador + plataforma.	Cumplido	Flujo E2E ejecutado en <i>testnet</i> .
Implementar verificación <i>on-chain</i> en puerta vía rol STAFF/Verificador.	Cumplido	Phase 4; demo de redención.
Documentar el trabajo de grado conforme a los estándares académicos de la PUJ.	Cumplido	Seis documentos entregados y validados.
Desplegar al menos 8 eventos de prueba en <i>testnet</i> .	Superado	12/12 eventos desplegados.

0.11.3. Lecciones aprendidas globales

Más allá de las lecciones puntuales por fase (Sección 0.8.3), el equipo identifica las siguientes lecciones **transversales** al proyecto:

- **Web2.5 como propuesta de valor:** permitir al usuario operar sin billetera y luego *opt-in* a la capa on-chain redujo enormemente la fricción de adopción y simplificó las pruebas de usabilidad con personas no técnicas.
- **Patrón XDR proxy *stateless*:** desacoplar la construcción de transacciones (backend) de la firma (cliente con Freightier) eliminó la necesidad de custodiar llaves y simplificó la superficie de auditoría.
- **Indexer como puente:** mantener un indexer sincronizando estado on-chain → PostgreSQL fue clave para servir vistas rápidas al frontend sin sacrificar la fuente de verdad criptográfica.
- **Kanban con WIP limitado** resultó más adecuado que sprints fijos para un equipo de cinco con horarios académicos heterogéneos.
- **Revisiones semanales con el director:** actuaron como mecanismo de control de alcance y validación temprana; el desplazamiento de un solo hito se atribuye en parte a este ritmo de retroalimentación.
- **Documentación viva:** actualizar el SPMP, el SRS y el SDD al final de cada fase (no solo al inicio) evitó la divergencia entre documento entregado y sistema construido.
- **Curva de aprendizaje de Soroban/Rust:** subestimada inicialmente; en proyectos futuros conviene reservar un *spike* técnico explícito antes de comprometer fechas de entrega.

0.11.4. Riesgos residuales y trabajo futuro

El prototipo entregado funciona en *testnet* y cumple los criterios de aceptación académicos. Para una eventual evolución hacia un producto operativo se identifican los siguientes **riesgos residuales** y **líneas de trabajo futuro**:

- **Migración a *mainnet***: requiere auditoría externa de los tres contratos Soroban, plan de manejo de incidentes y procedimientos de actualización (*upgradeability*) antes de mover valor real.
- **Pasarela de pagos real**: el *checkout* fiat es simulado; integrar una pasarela (Stripe, PayU u otra) implica cumplimiento PCI-DSS y rediseño del flujo de conciliación con on-chain.
- **Escalabilidad del indexer**: el indexer actual opera con *polling* cada 5 s sobre el RPC de Soroban; para volúmenes mayores conviene migrar a un esquema basado en *webhooks* o a un servicio gestionado.
- **Experiencia de usuario sin billetera**: aunque la capa Web3 es opcional, una eventual versión de producción debe ofrecer custodia opcional (billetera gestionada) para usuarios que prefieran no instalar Freight.
- **Internacionalización y accesibilidad (WCAG)**: ambos quedaron fuera de alcance del prototipo académico y deben abordarse antes de un lanzamiento público.
- **Modelo de negocio sostenible**: definir la estructura de comisiones (organizador / plataforma / reventa) en función de los costos operativos reales de *mainnet* y de pasarela de pagos.

0.11.5. Cierre formal del SPMP

Con la entrega de la versión **1.5** de este Plan de Gestión del Proyecto (21 de mayo de 2026) y de los demás artefactos académicos asociados, el equipo declara **cerrado** el ciclo de gestión del trabajo de grado SecureTicket correspondiente al semestre 2026-1.

- **Estado del repositorio**: congelado en la rama *main*; *tag* de cierre **v1.0-tg-cierre** aplicado el 24 de mayo de 2026.
- **Estado de la infraestructura**: servicios desplegados (Vercel, Railway, Supabase, contratos en *testnet*) se mantienen activos para la sustentación; su desmantelamiento posterior queda a discreción del equipo y del director.
- **Custodia documental**: los seis documentos del trabajo de grado se entregan a la biblioteca de la Pontificia Universidad Javeriana conforme al Formato de Biblioteca.

- **Responsabilidad post-cierre:** el equipo no compromete soporte ni mantenimiento posterior a la sustentación; cualquier evolución posterior se considera fuera del alcance del trabajo de grado.

Este Plan deja registro trazable y medible del proceso ejecutado, en concordancia con IEEE Std 1058-1998 y con las directrices del curso de *Planeación Proyecto Final* de la Pontificia Universidad Javeriana.

0.12. Referencias

- [1] IEEE Std 1058-1998: *IEEE Standard for Software Project Management Plans*.
- [2] ISO/IEC/IEEE 12207:2017: *Systems and software engineering — Software life cycle processes*.
- [3] Project Management Institute (PMI): *A Guide to the Project Management Body of Knowledge (PMBOK® Guide)*, 7th Edition, 2021.
- [4] Guía y plantilla del curso para SPMP, Pontificia Universidad Javeriana, 2025.
- [5] Stellar Development Foundation: *Stellar Documentation*, <https://developers.stellar.org/>.
- [6] Soroban Documentation: *Smart Contracts on Stellar*, <https://soroban.stellar.org/>.